agent: devin-local session: complete-pamphlet created: 2026-11-07T15:54:30Z

updated: 2026-11-08

Astart — TODO

The living backlog: open work only, in 14 fixed chapters. Done tasks and history are in `docs/TODO-archive.md`.

Dashboard (generated)

154 of 342 tasks done (45.0 %). 21 open tasks are ready to start; 167 still need a recipe. Regenerate with `python scripts/todo_index.py docs/TODO.md`.

#	Chapter	Done	Open	Ready	P0	P1
1	Foundations and measurement	27	1	1	0	0
2	Object-oriented code	5	10	3	1	9
3	Whole-project analysis	9	6	0	0	3
4	Precise types	7	13	0	0	10
5	Protocols and generics	9	16	0	2	8
6	Keyword arguments	6	7	0	0	6
7	Speed and shipping	2	14	1	0	3
8	Ontology and inference core	8	29	5	1	11
9	Cross-cutting engineering	2	6	1	0	6
10	Contract negotiation	19	25	1	5	18
11	Untyped to typed conversion	1	32	9	5	17
12	Typer and small tasks	59	17	0	0	0
13	Handoff batch	0	12	0	3	7

Open P0

ID	Readiness	Task
M2-13	needs recipe	M2 exit gate
M5-13	needs recipe	E6-verified synthesized protocols
M5-25	needs recipe	M5 exit gate
M8-21	needs recipe	M8 exit gate
F-17	needs recipe	Seeds from existing evidence, origin-tagged: default literal (`default`), proven…
F-18	ready	Acceptance policy `negotiation/policy.py` : settled type + independent origin + no open…
F-20	needs recipe	Behind `FixSettings.negotiate` (CLI `--negotiate`), let the negotiator replace the…
F-28	needs recipe	Determinism and idempotence: hash-seed test (as Q-15), `fix` ∘ `fix` byte-equal on both…
F-29	needs recipe	Gate: refresh `baseline.json` and `baseline_ext.json` with negotiation on; accept only…
P-02	needs recipe	Variable annotations (`x: int = 1`) for module, class and function locals behind…
P-03	needs recipe	Return types that are "not provable" (127 of the pair's misses): returns through locals,…
P-08	needs recipe	The reverse direction, `astart untype` : lossless removal of parameter, return and…
W-01	ready	Record the verification commands and their baseline results
W-02	ready	Find why the checker gate keeps only 4 annotations on the resolver pair
H-01	needs recipe	Reporters in the CLI (M7-09, Q-28, part of M7-16)
H-02	needs recipe	`[tool.astart]` in the CLI (M7-07 user-visible half, M7-16)
H-08	needs recipe	`astart untype` (P-08): the reverse of `fix`

Open P1

ID	Readiness	Task
M2-02	ready	Constructor/attribute resolution
M2-05	needs recipe	Rules-layer `self.X → ContractIndex` fallback
M2-06	ready	R9: attribute types
M2-07	needs recipe	Override awareness (E1)
M2-08	ready	Escape analysis v2
M2-09	needs recipe	Flow-sensitive locals
M2-10	needs recipe	Kwargs for constructors
M2-11	needs recipe	Kwargs for method calls
M2-12	needs recipe	Kwargs for bound-method callbacks
M3-07	needs recipe	Canonical public import locations
M3-09	needs recipe	`TYPE_CHECKING` placement
M3-13	needs recipe	`ensure_names` everywhere + expanded `known_imports`
M4-01	needs recipe	Prover: comprehensions, subscripts on typed names, builtin-method result table
M4-02	needs recipe	Prover: `Await` unwrap, `Try` / `Match` / `AugAssign` /loop handling
M4-03	needs recipe	Prover: yield typing + once-bound `ClassDef` calls
M4-06	needs recipe	R3: `generator` / `async`
M4-07	needs recipe	R4: `usage_abc`
M4-08	needs recipe	R5: `typeguard` / `TypeIs`
M4-11	needs recipe	R6: `union_sites`
M4-14	needs recipe	Variable/attribute annotations
M4-16	needs recipe	Accept-general/return-specific

M4-17	needs recipe	`Callable[[…], R]` params
M5-04	needs recipe	Generator/CM/indexing/operator/truthiness/ `len` members
M5-06	needs recipe	Read-only vs writable attribute members
M5-09	needs recipe	P2/P3: reuse-before-create
M5-10	needs recipe	Merge into shared `_protocols.py` + name lockfile
M5-16	needs recipe	Multi-variable relations
M5-19	needs recipe	T5: ParamSpec decorator forwarding
M5-22	needs recipe	T1: BoolOp/comparison shared `T`
M5-24	needs recipe	No user-symbol shadowing; project-wide declaration reuse
M6-02	needs recipe	Positional-only/keyword-only end-to-end
M6-05	needs recipe	`calls.missing-argument` detection rule
M6-06	needs recipe	`calls.unexpected-argument` detection rule
M6-07	needs recipe	`calls.ambiguous-callee` detection rule
M6-08	needs recipe	`calls.unresolved-callee` detection rule
M6-09	needs recipe	Idempotence with ruff line-wrapping
M7-01	needs recipe	Kill superlinear costs
M7-02	needs recipe	Perf budgets
M7-16	ready	Wire the built-but-unreachable product surface (audit finding)
M8-01	needs recipe	`imports.bare-import` IMP001
M8-02	needs recipe	`typing.missing-variable-annotation` TYP005
M8-03	needs recipe	`typing.no-object-annotation` TYP002
M8-04	needs recipe	`suppression.marker-comment` SUP001
M8-05	needs recipe	`parsing.module-does-not-parse` PRS001
M8-06	needs recipe	`typing.return-type-mismatch` / `incompatible-default`
M8-07	needs recipe	Docs family DOC001/003/004/006
M8-11	needs recipe	D6 remainder: `Exists` evaluation
M8-12	needs recipe	D8: procedural rules
M8-22	ready	Register the standalone detectors as catalog rules (audit finding)
M8-27	ready	IMP001 fixer: bare `import a` → `from a import b` + use-site rewrite
CC-01	needs recipe	Confidence calibration
CC-02	needs recipe	Determinism audit
CC-03	needs recipe	Error containment v2
CC-04	needs recipe	Property + mutation testing
CC-06	needs recipe	Convention guard
CC-07	ready	Commit hygiene (audit finding)
F-07	needs recipe	Complete the transfer table, forward *and* backward, per relation: key/value (`dict[K,…
F-13	needs recipe	Builder: capabilities
F-15	needs recipe	Builder: library calls
F-16	needs recipe	Builder: declared annotations, base-class and override signatures (E1, M2-07) as…
F-21	needs recipe	Returns as union contracts: join of all exits subsumes `_join_types` ; composes with…
F-22	needs recipe	Backward reconciliation of parameters: use-site capabilities/narrowing intersect the…
F-23	needs recipe	Conflict reporting: a detection rule (`typing.type-conflict` , info) that lists the…
F-24	needs recipe	Calibrate the lossy collapse with the X-25 method (~27% exact, 0 contradictions, n…

F-24	needs recipe	Calibrate the lossy collapses with the X-05 method (≥ 97 % exact, 0 contradictions, n ≥…
F-26	needs recipe	Explanation trail per candidate (origin, path, relation, distance) kept through merges;…
F-27	needs recipe	Performance: graph build linear in nodes; per-function incremental rebuild; budget tests…
F-30	needs recipe	`docs/negotiation.md` (model, adaptation rule, how to add a contract builder or a…
F-31	needs recipe	Gold family `negotiated` (≥ 60 cases: cycles, containers, narrowing, conflicts, unions,…
F-33	needs recipe	Cross-module edges through `ProjectIndex` (argument/return/attribute contracts across…
F-34	needs recipe	Safety bounds: per-module event and wall-clock budget; on overrun fall back to the…
F-37	needs recipe	Default-literal parameters in the negotiator (the last big shadow gap: 70 annotations, 60…
F-40	needs recipe	Comprehensions as derived nodes (the loop variable typed from the iterable's element…
F-41	needs recipe	Calls on builtin-typed receivers (`value.strip()` , `items.copy()` , `d.get(k)`) as derived…
F-43	needs recipe	Quote instead of refusing: where a negotiated annotation names a class not yet bound (`->…
P-04	needs recipe	"Tainted" parameters (83): types that only usage reveals (`x + 1` , `x.upper()` , `for _ in…
P-05	needs recipe	"No candidate" (72): the inference engine proposes nothing; measure which shapes and…
P-06	needs recipe	"Call sites unknown / uncorroborated / escapes" (130): fixtures are closed worlds; add…
P-07	needs recipe	Name-agnostic scoring of synthesized Protocols and TypeVars (the author's…
P-09	needs recipe	Round-trip gate: `fix(untype(typed))` against `typed` per pair, baseline stored, gate…
P-10	needs recipe	Import parity: every name the written annotations use is imported (M3-13), with the typed…
T-01	needs recipe	Dunder parameters from the language (`autotyping --annotate-magics`):…
T-02	needs recipe	Copy annotations from the base-class method of the same name (`infer-types` ; this is…
T-03	needs recipe	Apply a sibling stub (`retype` , which is unmaintained since 2022): when `foo.pyi` sits…
T-08	ready	Library return types from typeshed (`typeshed_client` , which `scripts/generate_libkw.py` …
W-03	ready	Stop `object` leaking into `--engine` annotations
W-04	ready	Remove the last `typeshed_client` use from `generate_libkw.py`
W-06	ready	Burn down unprovable calls in the negotiator
W-07	ready	Burn down unbindable calls and escaping functions
W-08	ready	Let the negotiator write default-literal parameters (F-37)
W-11	needs recipe	Annotate a method that returns its own class (`Self` or quoted name)
W-12	needs recipe	Annotate instance attributes from negotiation
H-03	needs recipe	Baselines in the CLI (M7-08)
H-04	needs recipe	Catalog rules TYP002 and TYP006 (Q-33, Q-34, M8-03)
H-05	needs recipe	PRS001 for files that do not parse (Q-35, M8-05): one finding at line 1 instead of an…
H-06	needs recipe	Returns / Raises docstring stubs (Q-61, M8-07)
H-07	needs recipe	Wider `known_imports` (M3-13)
H-09	needs recipe	Gate for the conversion pair (P-09)
H-11	needs recipe	`astart fix --jobs N` (X-06, M7-04/05): one process per file in `cli/commands.py` with…

START HERE — read this before touching anything

This section is for an agent that has not seen the project. It is short on purpose. Follow it literally; if something here is wrong, fix this section.

What the project is

`astart` is a lossless Python source fixer. `astart fix` adds missing keyword arguments, type annotations, imports and docstrings. Its rule is **zero wrong annotations**, so a type is written only when evidence proves it. Two extra modes trade that safety for coverage: `--annotate-variables` and `--engine` (conversion mode). The goal that drives current work: turn `dev/untyped.py` into `dev/typed.py` (and the same for `tests/pairs/resolver_extended_*`) automatically. Today it recovers about 13 % exactly in the safe mode and about 52–59 % in `--engine` mode; the old converter this project descends from reached 70

% with ~20 % wrong

% with ~20 % wrong.

The five commands you will use

```
# 1. fast tests (~70 s, 4 838 pass, 2 xfail). Run before and after a change.
python -m pytest -m "not slow" -q -p no:cacheprovider

# 2. lint ONLY the files you created or edited, in this order
ruff check --fix FILES ; ruff format FILES ; ruff check FILES --output-format concise

# 3. how far is the conversion? (numbers to compare are in "Current numbers")
PYTHONPATH=src:. python -m benchmarks.pair dev/untyped.py dev/typed.py
PYTHONPATH=src:. python -m benchmarks.pair dev/untyped.py dev/typed.py --engine --min-confidence 0.5

# 4. why is a node unknown?  (the burn-down tool: fix the biggest reason, rerun)
PYTHONPATH=src python scripts/opacity_report.py dev/untyped.py

# 5. which written annotations disagree with the author's?
PYTHONPATH=src:. python scripts/pair_wrong.py dev/untyped.py dev/typed.py --engine --min-confidence 0.5
```

Long jobs (more than ~100 s) must run in the background and be polled with a loop that sleeps at most 10 s per step and stops by 590 s; a foreground command over 600 s is killed. Slow tier: `python -m pytest -m slow -q -p no:cacheprovider` takes ~19 min and must pass (870 tests) before a task that changes `hints.py`, `facts.py`, `appliers.py`, `builders.py` or `methods.py` is called done.

House rules (a test and the user enforce these)

1. Never run `exactus`. Never add `# noqa` or `# type: ignore`.
2. Calls use keywords everywhere the callee allows it: `f(x=1)`, `parse(source=s)`, `walk(node=n)`. Signatures are keyword-only (`def f(*, x: int)`).
3. Google-style docstrings with `Args:` / `Returns:` on every function.
4. Module constants live in `@final class Constants` with `Final` attributes.
5. No `Any`, no `object` in annotations. Import names (`from ast import walk`), never `import ast`. Line length 79.
6. macOS: `sed -i ''` needs the empty string; prefer a Python snippet for edits.
7. Edit `docs/TODO.md` atomically; after changing a task's Status, Priority or Readiness run `python scripts/todo_index.py docs/TODO.md` (it only rewrites the dashboard block). Another agent works in the same tree: touch only the files your task names.

Where things live

Path	What
`src/astart/rules/hints.py`	the older evidence pipeline (`HintIndex`); do not rewrite
`src/astart/rules/negotiation/`	contract negotiation: `builders.py` (AST → graph), `negotiator.py` (solver), `policy.py` (what may be written), `transfer.py`, `containers.py`, `narrowing.py`, `supplement.py`, `variables.py`, `run.py`
`src/astart/rules/engine_hints.py`	conversion mode (`--engine`): the old converter's annotations as lowest-priority hints
`src/astart/stubs/`	our typeshed reader: `locate`, `versions`, `conditions`, `parser`, `resolver`, `returns` (migrated ideas of `typeshed_client`; no dependency)
`src/astart/_graph/`	the old converter ("the engine")
`src/astart/rules/catalog/*.json`	rules and fixes; edit by hand, `json.dumps(d, indent=2, sort_keys=True) + "\n"`
`benchmarks/` and `scripts/`	measurement; `pair.py`, `shadow.py`, recovery + gates
`docs/negotiation.md`	the model and the soundness rules; read it before editing `builders.py`

Current numbers (2026-11-08; a change must not lower the first four)

Metric	Value

Measure	Value
stdlib baseline (`benchmarks/baseline.json`)	42 / 324 correct, 0 wrong
external baseline (`benchmarks/baseline_ext.json`)	2 511 / 12 789 correct, 6 wrong, precision 99.77 %
gold superset test	negotiation changes 0 written annotations, adds 24 reviewed
semantic equivalence	15 / 15 projects (needs the 5 installs in W-01 step 4)
dev pair, safe mode	52 / 482 parameters+returns, 55 / 335 variables, 2 wrong
dev pair, `--engine`	253 / 482, 174 / 335, ~104 wrong
resolver pair, safe mode	113 / 557, 59 / 329, 2 wrong
resolver pair, `--engine`	329 / 557, 185 / 329, ~94 wrong
old converter's own output vs resolver twin	388 / 557, 207 / 329, ~106 wrong

Definition of done for any task

1. The new behaviour has a test that **fails without the change** (prove it: disable the change, watch the test fail, re-enable).
2. The fast tier passes; the lint command is clean on your files.
3. If the task touches inference or fixing: run the pair command and write the before/after numbers in the task's evidence line.
4. If it can change what is written: run the shadow comparison and the two recovery gates (commands in W-01); both must print `PASS` .
5. Tick the box, add a two-line evidence note under the task, stop.

Decisions already made (do not reopen them)

- Confidence only orders candidates; acceptance is decided by evidence class (`policy.accept`). The inference engine is corroboration in the safe mode and the lowest tier in `--engine` mode.
- Unknown is not absence: a node that lost a contribution or has no evidence is *opaque* and everything downstream is refused (`docs/negotiation.md`).
- No new runtime dependencies. Ideas from other projects are re-implemented.
- A negotiated annotation never names a class that does not exist yet when the `def` runs (`bound_when_evaluated`).
- `--engine` must stay opt-in; the default stays "zero wrong".

Execution map — measurement first

This map is a planning decision, not new completion evidence. It organizes the task owners into deliverable work packages matching milestones M0–M8. Sequencing is fixed by the evidence policy: nothing may be claimed more precise than the M1 harness can measure, and no guarded fix is trusted without the E6 checker gate.

Sequence	Work package and owning IDs	Prerequisites	Smallest useful delivery and stop condition
0, done	Trustworthy slice: M0-01…M0-10	—	Defects D1–D3, D6, D9 fixed; exactus green; baseline committed.
1, current	Measured: M1-01…M1-18	M0	Dashboard + baseline published; E6 gate rejects real regressions; D7 enforced. Stop claiming precision/recall until measured.
2	Object-oriented code works: M2-01…M2-13	M1 oracle	Method recall ≥3× baseline at unchanged precision; D4/D5 closed.
3	Whole project: M3-01…M3-14	M2	Cross-module recall met; canonical imports; 100k-line repo in budget.
4	Precise types: M4-01…M4-19	M1, M3	Corpus recall step-change at ≥99.5% guarded precision.
5	Structural and generic: M5-01…M5-25	M4	Protocol/TypeVar gold cases green; E6-verified synthesis.
6	Kwargs bulletproof: M6-01…M6-09	M3 library table	Danger matrix green; detection rules shipping; idempotent under ruff.
7	Fast and shippable: M7-01…M7-15	M3–M6	≤5 s/12k lines; `[tool.astart]` surface; reporters + LSP.
8	Whole ontology + inference core: M8-01…M8-21	M4, M6	`verify_catalog` all invariants; spec-§5 rule set present.

Slice handoff contract

Before coding, add ordinary prose beneath the selected task (under **Context**): `Scope` , `Depends on` , `Source` , `Acceptance` , `Evidence` , `Known limits` — populated with exact behavior and inspected files, not generic promises. Success, invalid input, refusal, lost-checker and non-idempotent paths all need named proof, not a happy path. At completion update only the satisfied obligation; if a parent still lacks gold cases, checker-gate or corpus evidence, keep it open and record the accepted slice.

Decisions (all locked with the user)

Decisions (all locked with the user)

- **Scope: full roadmap M0–M8** — including `_graph` / `semantic` inference-core bug fixes.
- **Union policy: enabled (guarded)** — `T1 | T2 [| …]` accepted when ≤3 distinct definite spellings and inference corroborates.
- **TypeVar style: classic default + version-aware PEP 695** — `T = TypeVar("T")` default; `def f[T](x: T)` when `requires-python >= 3.12`.
- **Checker oracle: yes, optional auto-detect** — guarded fixes gated on pyright/basedpyright when installed (E6); without it, guarded fixes need a higher internal evidence level.
- **Default aggressiveness: configurable per-profile** — profiles/ `[tool.astart]` set the safety ceiling; default profile applies proven-only, guarded behind opt-in flag/profile.
- **Protocol placement: shared `_protocols.py` + name lockfile** — same-member protocols merged project-wide with deterministic stable names; config switch preserves per-function placement.
- **Git init: yes** — initialize a repository in M0 so every milestone is diff-reviewable and rollback-safe.
- **Library keyword table: stdlib first, extensible** — curated typeshed-generated table; third-party packages join only via explicit allow-list.

North star (roadmap §1)

`astart fix` takes untyped, positionally-called Python and returns the same program with: (1) every safely keyword-capable call keyword-called; (2) every parameter/return/attribute/variable annotated with the **most precise type the evidence supports**; (3) `Protocol` s and type variables synthesized where structural or generic; (4) every required import; (5) formatting untouched except one final `ruff format`. Success = **zero wrong annotations at the highest recall precision allows**, proven by an independent checker, not by astart's own opinion.

Non-negotiables: behaviour preserving · never invalid Python · comments/layout kept · idempotent · explained skips (closed enum) · independent verification (E6 gate).

Accuracy model (adopted wholesale)

Evidence ladder: **E0** syntax-proves · **E1** declared-elsewhere (stubs, base signatures, `@overload`, implemented ABCs) · **E2** closed-world usage (all in-module sites, no escape) · **E3** cross-module call sites · **E4** library signature flow · **E5** inference proposal (candidate only, never alone) · **E6** external checker does not object.

Policy: a guarded fix needs (E2 ∨ E3 ∨ E4) ∧ (E5 agrees ∨ E0 proves) ∧ E6 does not object. `proven` = E0/E1 only. `rejected-by-checker` is a reported outcome, never silent. Confidence calibrated against benchmarks; `--min-confidence` knob. Checker auto-detected; absent → guarded fixes need higher internal evidence.

Task template

Every task has exactly this shape (the dashboard parses it):

```
### F-17 — Imperative title without a final period
- **Status:** open            <!-- open | done -->
- **Priority:** P0            <!-- P0 | P1 | P2 | P3 | Untriaged -->
- **Area:** Backend           <!-- Backend | Testing / QA | Documentation | DevOps / Release | Uncategorized -->
- **Readiness:** needs recipe <!-- ready | needs recipe | done -->

Why / context in one or two sentences.

**Facts:** files and line references.

**Steps:**
1. One edit or one command, with its check.

**Done when:** one command and the number it must print.
```

- `ready` means the task has numbered steps or a **Done when** line. `needs recipe` means the task still shows `_to be written_` under **Steps** and **Done when**: a planner writes them first (one edit or one command per step, with the file and the check), then sets **Readiness** to `ready` and reruns the dashboard script.
- Finish a task: set **Status** to `done` and **Readiness** to `done`, then run `python scripts/todo_index.py docs/TODO.md`. Do not delete the task and do not change its ID.
- Priorities: P0 blocks a trustworthy release; P1 is required core work; P2 is secondary; P3 is optional polish.

1. Foundations and measurement

Scope: Checker oracle, benchmarks, gold sets: how we know a fix is right.
ID prefixes: M0, M1

M1-10 — Throughput benchmarks

- **Status:** open
- **Priority:** P2
- **Area:** Testing / QA
- **Readiness:** ready

Context: Runtime by size bucket (500/2k/12k/50k lines); current 12k-lines ≈ 47 s is the regression baseline.

Acceptance: bucket table published; superlinear growth flagged.

Audit 2026-11-07 (partial): `docs/benchmarks.md` now carries the bucket table (~1.2k/2.2k/12k measured via `benchmarks/profile_perf.py --json`) and flags superlinear `check` growth; the ~50k row is pending.

Completed

ID	Task	Priority	Area
M0-01	Git init + ruff-format drift policy	P0	DevOps / Release
M0-02	D1: `ensure_names` in `add_inferred_annotation`	P0	Backend
M0-03	D2: `'Unknown'` is non-definite evidence	P0	Backend
M0-04	D3: imported constructors type their calls	P0	Backend
M0-05	D6: `Exists` crash guarded	P0	Backend
M0-06	D9: `typing.add_noreturn` fix	P0	Backend
M0-07	Final-review findings C1, C2, I1–I6	P1	Backend
M0-08	Type-policy cleanup	P1	Backend
M0-09	D10 suite-flakiness investigation	P1	Testing / QA
M0-10	Bare `exactus` clean	P0	Testing / QA
M1-01	Closed skip/rejection reason enums	P0	Backend
M1-02	E6 checker oracle, core	P0	Backend
M1-03	E6 oracle remainder	P0	Backend
M1-04	Benchmark harness scaffolding	P0	Testing / QA
M1-05	Annotation-recovery benchmark	P0	Testing / QA
M1-06	Gold set expansion 34 → ≥500	P1	Testing / QA
M1-07	Corpus expansion 20 → ≥100 real modules	P1	Testing / QA
M1-08	Precision/recall dashboard	P0	Testing / QA
M1-09	Checker-delta metric	P1	Testing / QA
M1-11	Stability benchmarks	P1	Testing / QA
M1-12	Semantic-equivalence runs	P2	Testing / QA
M1-13	Writer fuzzing	P1	Testing / QA
M1-14	Full-CLI idempotence tests	P1	Testing / QA
M1-15	D7: real `FixOffer.confidence` + `min_confidence` enforcement	P0	Backend
M1-16	Crash containment for analyzer failures (audit finding)	P0	Backend
M1-17	Gold set at fix level	P1	Testing / QA
M1-18	Fast test tier	P2	Testing / QA

2. Object-oriented code

Scope: Classes, methods, attributes, constructors and overrides.
ID prefixes: M2

M2-13 — M2 exit gate

- **Status:** open
- **Priority:** P0
- **Area:** Testing / QA
- **Readiness:** needs recipe

Context: method recall ≥3× M1 baseline at unchanged precision, measured on the benchmark dashboard.

Audit 2026-11-08: still not done — benchmarks track overall precision/recall only; no method-recall metric or ≥3× gate.

Steps: *to be written*

Done when: *to be written*

M2-02 — Constructor/attribute resolution

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** ready

Context: `obj.m` when class known, `Class.m`, `super().m`, `cls()`, `@classmethod` / `@staticmethod`, MRO walk.

Acceptance: each receiver form resolves to the defining signature.

Partial (open): `obj.m` (obj assigned once from `C()`), `Class.m(obj, …)` (explicit receiver stays positional: `facts.kept_positionals`), `C().m` and `self.m` (single-base chain) resolve. Missing: `super().m`, `cls()` / `cls.m`, `@classmethod` / `@staticmethod` through `self`, true MRO for multiple bases.

Audit 2026-11-08 (partial): `self.m`, `C().m`, `obj.m`, `Class.m` and static/class methods via the class keyword; `super()`, `cls()`, `self`-dispatched decorated methods and true MRO still refused (`_find` requires a single in-module base).

M2-05 — Rules-layer `self.X → ContractIndex` fallback

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: so attributes assigned in `__init__` resolve without full inference.

Audit 2026-11-08 (partial): `rules/attributes.py` resolves agreed `self.x` types from all writes without full inference and feeds `HintIndex` / `provable._attribute`; but nothing feeds `ContractIndex` /call binding — `self.attr(...)` calls and attribute entries in call-site envs are unresolved.

Steps: *to be written*

Done when: *to be written*

M2-06 — R9: attribute types

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** ready

Context: `self._attr` assignments across methods → class-level annotations; `Final` / `ClassVar` when all assignments agree; untaint `self._attr` returns.

Acceptance: `self._x: int = …` emitted once at class level, not per method.

Partial (2026-11-07, agent: claude): `rules/attributes.py` agrees all `self.x = …` writes of a class (literals, annotated parameters, shared across methods; refuses on conflicts, dynamic markers, `__setattr__`, slots, inheritance clashes) and rule `TYP005` / fix `typing.add-attribute-annotation` (guarded, conf 0.9) annotates the first write as `self.x: T = …`. `self.x` reads are now typed too: `attributes.Attributes.types` (agreed type per class) feeds `HintIndex._judge_derived` as `"self.x"` entries in the typer's environment, the unrebound receiver stops being a taint origin, and the inference candidate must still agree (or equal the proof minus `None`). Open: class-level placement, `Final` / `ClassVar`; the module-wide `_external_stores` name count blocks two classes that write the same attribute name. Tests: `tests/test_rules_attributes.py`, `tests/test_rules_attribute_reads.py`.

M2-07 — Override awareness (E1)

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: Overrides inherit base signatures; fixes never contradict the base; keyword fixes use base parameter names.

Partial (open): an attribute call is trusted only if every definition of that name in the module takes identical parameters and no `obj.m = …` store exists (`MethodIndex.is_safe`). Missing: inheriting the base signature for annotation, and keywording through differently-named overrides.

Steps: *to be written*

Done when: *to be written*

M2-08 — Escape analysis v2

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** ready

Context: Real "used as value": callbacks, `functools.partial`, `map`, decorators, registries, `getattr`, attribute escapes — replaces today's coarse `evidence.escaped`.

Acceptance: `Rejection.escapes` fires only on true escapes; corpus shows fewer uncorroborated refusals.

Audit 2026-11-08: still not done — `hints._escaped_names` remains the coarse "any non-callee Load escapes" heuristic; no callback/`partial`/`map`/decorator/`getattr`/registry handling. Project open-world evidence (`ProjectView.open_world`) narrows exported names only.

M2-09 — Flow-sensitive locals

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: `isinstance`/`is None`/truthiness/ early-return/`assert` narrowing, merge joins, loop widening to fixpoint.

Audit 2026-11-08 (partial): `type_inference` has isinstance/`is None`/TypeGuard/TypeIs/`not` narrowing, if-merge joins and a worklist solver; missing truthiness narrowing, early-return unreachability and a true loop-widening fixpoint. Progress 2026-11-08 (agent: claude): `provable._flow_returns` narrows Optional parameters (`x: T | None`) at `return` statements: both arms of `x is None` / `x is not None`, and the code after an arm that cannot fall through (early return) — and `isinstance(x, C | (C, D))` / `not` of either on any flat union (`bool` among the arms blocks `int` tests); a rebound or reassigned parameter is never narrowed. A flow proof that is a strict part of the engine's union is accepted (`hints._refines`). Non-guaranteed proofs still need the inference candidate to agree. Also (2026-11-08): `assert <test>` narrows what follows; truthiness (`if x:` / `if not x:`) removes `None` on the true arm only (the false arm keeps the full union, since `0` is falsy too); the `if x is None: x = v` defaulting idiom narrows `x` after the `if` when it is the parameter's only write and `v`'s proven type is one of the non-`None` arms (anything else disables narrowing for that name). Typer additions: `str.split/rsplit/ splitlines` → `list[str]`, `encode`/`decode`. Open: loop widening, narrowing for call-site evidence and attribute reads, `match` statements, `try`/`except` joins. Tests: `tests/test_rules_flow_narrowing.py`.

Steps: *to be written*

Done when: *to be written*

M2-10 — Kwargs for constructors

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: (7.2): dataclass/NamedTuple/attrs/pydantic.

Audit 2026-11-08 (partial): `__init__` classes, plain dataclasses (incl. inherited fields) and class-syntax NamedTuple bind; attrs/pydantic, `collections.namedtuple`, dataclass `kw_only`/`InitVar`/`init=False`/`__new__`-only still refused.

Steps: *to be written*

Done when: *to be written*

M2-11 — Kwargs for method calls

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: (7.3) once M2-01…03 land.

Partial (open): instance-method calls keyword correctly via `self.`, `C().`, `obj.` and `Class.` receivers; waits on M2-02/M2-03 for `super()`, constructors and class/static methods.

Audit 2026-11-08 (partial): method kwargs work for `self.m`, `C().m`, bound `obj.m`, explicit-receiver `Class.m(obj,…)` and static/class via class; `super()`, `cls`, self-dispatched decorated methods remain refused.

Steps: *to be written*

Done when: *to be written*

M2-12 — Kwargs for bound-method callbacks

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: (7.4) incl. `functools.partial`.

Audit 2026-11-08: still not done — no `functools.partial` /callback/bound-method handling; danger matrix pins `p = partial(c.m, 1); p(2)` and `getattr(c,"m")(1,2)` as refused.

Steps: *to be written*

Done when: *to be written*

Completed

ID	Task	Priority	Area
M2-01	D5: `self.m()` binding	P0	Backend
M2-03	Instantiation → `__init__` / `__new__` /dataclass fields	P1	Backend
M2-04	D4: `trusted_binding` v2 for expression receivers	P0	Backend
M2-14	E1 for protocol dunders (audit finding)	P1	Backend
M2-15	Typed call-site arguments from their scope (new)	P1	Backend

3. Whole-project analysis

Scope: Evidence that crosses module boundaries.
ID prefixes: M3

M3-07 — Canonical public import locations

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: (`collections.abc.Sequence` not `typing.Sequence`; shortest `__all__`-honouring path).

Audit 2026-11-08 (partial): `imports.Constants.known_imports` maps the ABCs to `collections.abc` and `typevars` emits `collections.abc` spellings. Missing: shortest `__all__`-honouring path selection — nothing uses `SymbolTable.exports()` to pick import spellings, and project classes are never imported by synthesized annotations.

Steps: *to be written*

Done when: *to be written*

M3-09 — `TYPE_CHECKING` placement

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: for cycle/heavy-module safety + `from __future__ import annotations` where runtime consumers allow.

Audit 2026-11-08: nothing in the `rules/` fix path emits `if TYPE_CHECKING:` or `__future__` imports (existing blocks are only *read* for binding correctness). `_graph/conversion/synthesis.py` has a full `ImportPlacementPolicy` / `TYPE_CHECKING_BLOCK` engine but is not wired into `fix_source`.

Steps: *to be written*

Done when: *to be written*

M3-13 — `ensure_names` everywhere + expanded `known_imports`

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: (audit 2026-11-08: `known_imports` now has 26 entries incl. the ABCs, date/time, `Decimal` / `Fraction` / `UUID`, `Final` / `NoReturn` / `Optional` / `Protocol` / `Self` / `TypeVar` / `Union`; still missing `Never`, `TypeGuard` / `TypeIs`, `ClassVar`, `Literal`, `ParamSpec`, `Concatenate`, `TypeVarTuple`, `TypedDict`, `NamedTuple`, `Annotated`, `override`, `final`, `re.Pattern` / `Match`. `ensure_names` is called in only 3 applier sites — other emitters hand-roll `insert_import`.) full `collections.abc`, `NoReturn` / `Never`, `TypeGuard` / `TypeIs`, `Self`, `ClassVar`, `Literal`, `ParamSpec`, `Concatenate`, `TypeVarTuple`, `TypedDict`, `NamedTuple`, `Annotated`, `override`, `final`, `Decimal`, `Fraction`, `UUID`, date/time, `re.Pattern` / `Match`.

Steps: *to be written*

Done when: *to be written*

M3-08 — File-style matching

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** needs recipe

Context: for import emission.

Audit 2026-11-08: `insert_import` always emits `from X import name` (merged or after the leading import block); no file-style, aliasing or relative-vs-absolute matching.

Steps: *to be written*

Done when: *to be written*

M3-11 — Unused-import cleanup

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** needs recipe

Context: (own-added only).

Audit 2026-11-08: nothing removes imports the fixer added; no own-added tracking exists (`insert_import` returns only the local name).

Steps: *to be written*

Done when: *to be written*

M3-12 — Deterministic pre-ruff import placement

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** needs recipe

Audit 2026-11-08 (partial): placement is deterministic — after the docstring + contiguous leading imports, merged into existing `from` statements, AST-order `ensure_names`; idempotence tested. Missing: sorting/grouping (stdlib/third-party/local sections, sorted aliases); `ruff format` does not reorder imports.

Steps: *to be written*

Done when: *to be written*

Completed

ID	Task	Priority	Area

M3-01	Import resolution (done 2026-11-08)	P1	Backend
M3-02	Whole-project call graph (done 2026-11-08)	P1	Backend
M3-03	E3 cross-module call sites (done 2026-11-08)	P1	Backend
M3-04	E4 typeshed/ `py.typed` /stub-package loading (done 2026-11-08)	P1	Backend
M3-05	K1: curated "safe to keyword" library table	P1	Backend
M3-06	Version awareness (done 2026-11-08)	P1	Backend
M3-10	Shadowing/collision avoidance	P1	Backend
M3-14	M3 exit gate (done 2026-11-08)	P1	Testing / QA
M3-15	Wire the library keyword table into binding (audit finding)	P1	Backend

4. Precise types

Scope: The hint lattice and the prover: which types can be proven.
ID prefixes: M4

M4-01 — Prover: comprehensions, subscripts on typed names, builtin-method result table

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: (R11: comprehension returns untyped).

Audit 2026-11-08 (partial): comprehensions done in `provable._Typer`; `Subscript` and builtin-method result tables (`"".join` , `str.strip`) still absent from the prover.

Steps: *to be written*

Done when: *to be written*

M4-02 — Prover: `Await` unwrap, `Try` / `Match` / `AugAssign` /loop handling

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: (R7 coverage).

Partial (2026-11-07): `provable._type_rebound_locals` types locals bound by several plain/augmented assignments when every binding agrees on one type (induction for `total = 0; total += 1` , `x = x * 2`); loop, `with` , `except` , import, walrus and `del` bindings of the name block it. Still open: `Await` unwrap, `Try` / `Match` result typing, loop-carried widening. `tests/test_rules_multi_binding_locals.py` .

Audit 2026-11-08 (partial): `Try` fall-off and agreeing multi-bindings done; `Await` , `Match` and loop-target typing still unproven in the prover (loop targets excluded in `scope_types`).

Steps: *to be written*

Done when: *to be written*

M4-03 — Prover: yield typing + once-bound `ClassDef` calls

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: (extends D3).

Audit 2026-11-08: still not done — generators still refused (`Rejection.generator`); `C()` on a once-bound local class still unproven in the prover (semantic layer synthesizes constructors but hints re-judge with the prover result).

Steps: *to be written*

Done when: *to be written*

M4-06 — R3: `generator` / `async`

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: lattice members (replace the `Rejection.generator` refusal).

Audit 2026-11-08: still not done — no `Generator` / `AsyncGenerator` lattice member; `_graph` emits only imprecise `Generator[object]` shapes that cannot pass the hint gate.

Steps: *to be written*

Done when: *to be written*

M4-07 — R4: `usage_abc`

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: — usage → `Iterable` / `Sequence` / `Mapping` / `Sized` / `Container` / `Callable` / `Awaitable`, corroborated by `subtype()` over call-site types.

Audit 2026-11-08: still not done — no `usage_abc` basis; `_graph` usage observations emit `Iterable` / `Sequence` / `Mapping` / `Callable[..., object]` candidates that die at the exact-spelling call-site gate.

Steps: *to be written*

Done when: *to be written*

M4-08 — R5: `typeguard` / `TypeIs`

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: emission where provable.

Audit 2026-11-08: still not done — `TypeGuard` / `TypeIs` are consumed for narrowing only; no emission path (`_graph` templates have no callers).

Steps: *to be written*

Done when: *to be written*

M4-11 — R6: `union_sites`

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: — `T1 | T2 [| …]` when ≤3 distinct definite spellings and inference corroborates (locked guarded policy).

Audit 2026-11-08: still not done — `_join_types` only produces `T | None`; disagreeing call sites never become a union (`call_sites_disagree`). Progress 2026-11-08 (agent: claude): call-site unions — when every site is syntax-proven, there are 2–3 distinct non-`None` types and the inference candidate is exactly that set, `hints._site_union` accepts it (`HintBasis.union_sites`, confidence 0.9; `bool` beside `int` collapses to `int`). External corpus: no change (2462 correct, 8 wrong, gate PASS), so it is safe but dormant on library code. Tests: `tests/test_rules_union_sites.py`, gold family `call_sites_disagree`. Open: union return types, `None` members (`optional_default` already covers one), and the tower collapses from the old Semantic Contract Farm (`int | float → float`, `list | set → Collection`) — calibrate before enabling. Reference design for real forward/backward reconciliation: `semantic_contract_farm_project` (contracts with polarity/confidence, bidirectional propagation, evidence algebra, objective solver).

Steps: *to be written*

Done when: *to be written*

M4-14 — Variable/attribute annotations

- **Status:** open
- **Priority:** P1

- **Area:** Backend
- **Readiness:** needs recipe

Context: (`x: list[int] = []` , empty containers from later `append` s) — module/class level or checker-required only, never gratuitous locals.

Audit 2026-11-08 (partial): attribute annotations done (`attributes.py` + TYP005 catalog rule); variable annotations detector exists but is unwired (M8-22), has no applier, and the empty-container-from- `append` case is unhandled in the live path.

Steps: *to be written*

Done when: *to be written*

M4-16 — Accept-general/return-specific

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: — params → `Sequence` / `Iterable` / `Mapping` , returns → `list` / `dict` .

Audit 2026-11-08 (partial): narrow typevar element plans (`Sequence[T]` / `Iterable[T]` params, `list` / `tuple` returns) cover the pass-through idiom; no general accept-general/return-specific policy and no `Mapping` emission.

Steps: *to be written*

Done when: *to be written*

M4-17 — `Callable[[…], R]` params

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: from call shape + result use.

Audit 2026-11-08: still not done — `_graph` only refines pre-existing `Callable[..., R]` params; nothing annotates a bare parameter from call shape.

Steps: *to be written*

Done when: *to be written*

M4-13 — R12: `Self` / `ClassVar` / `Literal` / `TypeAlias` / `NewType`

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** needs recipe

Context: where provable.

Audit 2026-11-08 (partial): `Self` fully wired (`typevars` / `synthesize_self` , ≥3.11 gate); `ClassVar` / `Literal` / `TypeAlias` / `NewType` are parse/consume-only, never emitted.

Steps: *to be written*

Done when: *to be written*

M4-18 — Data-carrier synthesis

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** needs recipe

Context: (`TypedDict` / `NamedTuple` / `@dataclass`) — suggest-only initially.

Audit 2026-11-08: still not done — no carrier synthesis; existing TypedDict/NamedTuple/dataclass constructors are only consumed/refined.

Steps: *to be written*

Done when: *to be written*

M4-19 — `@overload` synthesis

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** needs recipe

Context: — suggest-only initially; each item gets own catalog fix + safety class + gold cases + E6 gate.

Audit 2026-11-08: still not done — only a hardcoded `dict.get` -override overload special case in `_graph` ; no general synthesis, catalog fix or gold family.

Steps: *to be written*

Done when: *to be written*

Completed

ID	Task	Priority	Area
M4-04	R1: `optional_default`	P1	Backend
M4-05	R2: `variadic_args`	P1	Backend
M4-09	`never_returns`	P1	Backend
M4-10	R10: `syntax_proven`	P1	Backend
M4-12	R8: expression defaults	P1	Backend
M4-15	Container precision	P1	Backend
M4-20	Taint rule is too conservative for argument-independent results (audit finding)	P1	Backend

5. Protocols and generics

Scope: Synthesised Protocols, TypeVars, ParamSpec.
ID prefixes: M5

M5-13 — E6-verified synthesized protocols

- **Status:** open
- **Priority:** P0
- **Area:** Testing / QA
- **Readiness:** needs recipe

Context: — every emitted protocol checked against actual call sites; widen or drop on objection.

Audit 2026-11-08 (partial): `typing.synthesize-protocol` is `guarded` and the runner runs the external checker on guarded rounds (`SkipCode.checker_rejected`) — drop-on-objection at whole-module granularity. Progress 2026-11-08 (agent: claude): `rules/protocol_verify.py` `call_site_objection` checks every direct call of the function and drops the synthesis when an argument provably lacks a required member (builtin value, `None` , or an instance of a plain in-module class with no bases/ decorators/dynamic lookup); anything undecidable is no objection. Wired into fact `parameter.has_structural_protocol` . Tests: `tests/test_rules_protocol_verify.py` . Open: the *widen* path (drop only the offending member instead of the whole Protocol) and member-signature (arity) checks against call sites.

Steps: *to be written*

Done when: *to be written*

M5-25 — M5 exit gate

- **Status:** open
- **Priority:** P0
- **Area:** Testing / QA
- **Readiness:** needs recipe

Context: protocol/typevar gold cases green; zero invalid output on the danger corpus.

Steps: *to be written*

Done when: *to be written*

M5-04 — Generator/CM/indexing/operator/truthiness/ `len` members

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: with correct dunder signatures.

Steps: *to be written*

Done when: *to be written*

M5-06 — Read-only vs writable attribute members

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: (read/write evidence).

Steps: *to be written*

Done when: *to be written*

M5-09 — P2/P3: reuse-before-create

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: — existing/typing/ `collections.abc` / `os` protocols (`SupportsRead` , `Iterable` , `Sized` , `PathLike` , `typing.IO`) with minimal-sufficient-member selection.

Steps: *to be written*

Done when: *to be written*

M5-10 — Merge into shared `_protocols.py` + name lockfile

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: (locked); deterministic stable names.

Steps: *to be written*

Done when: *to be written*

M5-16 — Multi-variable relations

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: — `pair(a,b) → list[T|U]` vs `list[T]` by call-site evidence; `dict[K,V]` .

Audit 2026-11-08 (partial): `multi_return` / `_synthesize_multi` emit `list[T | T2]` , `tuple[T, T2]` , `dict[T, T2]` with fresh per-param TypeVars; the call-site-evidence half (`list[T|U]` vs `list[T]`) is absent — unions are always spelled for distinct parameters.

Steps: *to be written*

Done when: *to be written*

M5-19 — T5: ParamSpec decorator forwarding

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: — `f: Callable[P,R] → Callable[P,R]`.

Steps: *to be written*

Done when: *to be written*

M5-22 — T1: BoolOp/comparison shared `T`

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: — `a or b`, `a if a>b else b` (replaces the BoolOp passthrough refusal).

Audit 2026-11-08 (partial): BoolOp half done — `_is_pure` accepts `BoolOp` and `_Typer._boolop` requires one shared type, so `return x or x` → `-> T` while `x or 0` refuses. Comparison half open: `a if a > b else b` still refuses (test-position use needs an orderable bound; `Compare` not in `_is_pure`).

Steps: *to be written*

Done when: *to be written*

M5-24 — No user-symbol shadowing; project-wide declaration reuse

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: option (T2–T4 remainder).

Audit 2026-11-08 (partial): TypeVar names avoid module names, class type params and existing plain `TypeVar` declarations (reused); `insert_import` aliases on collision. Gaps: no cross-file/project-wide reuse (`fix_source` is per-module) and function-scope parameter names are not excluded — a sibling parameter literally named `T` could collide with an emitted `x: T`.

Steps: *to be written*

Done when: *to be written*

M5-07 — `runtime_checkable` only under `isinstance`

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** needs recipe

Context: usage evidence.

Steps: *to be written*

Done when: *to be written*

M5-11 — Config switch for per-function placement

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** needs recipe

Context: alternative.

Steps: *to be written*

Done when: *to be written*

M5-12 — `TYPE_CHECKING` placement

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** needs recipe

Context: for type-only protocols.

Steps: *to be written*

Done when: *to be written*

M5-20 — `TypeVarTuple` variadic passthrough

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** needs recipe

Steps: *to be written*

Done when: *to be written*

M5-21 — `T_co` / `T_contra` variance

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** needs recipe

Context: where provable.

Steps: *to be written*

Done when: *to be written*

M5-23 — PEP 695 `def f[T](x: T)`

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** needs recipe

Context: behind `requires-python >= 3.12` (locked style).

Steps: *to be written*

Done when: *to be written*

Completed

ID	Task	Priority	Area
M5-01	P1: typed protocol member returns	P1	Backend
M5-02	Typed protocol member params	P1	Backend
M5-03	`async def` protocol members	P1	Backend
M5-05	P4: `__call__` callback protocols	P1	Backend
M5-08	Collision-safe member-name allocation	P1	Backend
M5-14	Bound TypeVars	P1	Backend
M5-15	Constrained TypeVars	P1	Backend
M5-17	Element pass-through	P1	Backend
M5-18	`Self`	P1	Backend

6. Keyword arguments

Scope: Making `calls.add-keywords` sound and complete.
ID prefixes: M6

M6-02 — Positional-only/keyword-only end-to-end

- **Status:** open
- **Priority:** P1
- **Area:** Backend

- **Area:** Backend
- **Readiness:** needs recipe

Context: — `/`, `*`, `**kwargs` forwarding, `functools.partial`, `super().__init__` chains.

Audit 2026-11-08 (partial): posonly/kwonly/ `*args` / `**kwargs` / `partial` / `super()` rows pass in the 90-case danger matrix; gaps: no `super().__init__` row, no `**kwargs`-forwarding wrapper test, partial keywording undecided.

Steps: *to be written*

Done when: *to be written*

M6-05 — `calls.missing-argument` detection rule

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: (K4).

Audit 2026-11-07 (partial): `find_missing_arguments` in `rules/call_problems.py` flags trusted-binding `missing required argument` errors (functions and methods); tests in `tests/test_rules_call_problems.py`; not in the catalog (→ M8-22).

Audit 2026-11-08 (partial): `find_missing_arguments` + tests exist; not in `catalog/calls.json` — `check` never reports it (registration deferred to M8-22).

Steps: *to be written*

Done when: *to be written*

M6-06 — `calls.unexpected-argument` detection rule

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: (K4).

Audit 2026-11-07 (partial): `find_unexpected_arguments` in `rules/call_problems.py` flags `unexpected keyword arguments` and `too many positional arguments` binding errors; tests in `tests/test_rules_call_problems.py`; not in the catalog (→ M8-22).

Audit 2026-11-08 (partial): `find_unexpected_arguments` + tests exist; same catalog gap.

Steps: *to be written*

Done when: *to be written*

M6-07 — `calls.ambiguous-callee` detection rule

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: .

Audit 2026-11-07 (partial): `find_ambiguous_callees` in `rules/call_problems.py` flags `indeterminate` bindings (uncertain `*args` / `**kwargs` expansions); tests in `tests/test_rules_call_problems.py`; not in the catalog (→ M8-22).

Audit 2026-11-08 (partial): `find_ambiguous_callees` + tests exist; same catalog gap.

Steps: *to be written*

Done when: *to be written*

M6-08 — `calls.unresolved-callee` detection rule

- **Status:** open
- **Priority:** P1
- **Area:** Backend

- **Readiness:** needs recipe

Context: (K3 remainder: receiver expressions distrusted).

Audit 2026-11-07 (partial): `find_unresolved_callees` in `rules/call_problems.py` flags calls whose callee root is bound in the module but the trusted `CallBindings` layer refuses them (redefinitions, conditional defs, distrusted receivers); builtins and unbound names are out of scope; tests in `tests/test_rules_call_problems.py`; not in the catalog (→ M8-22).

Audit 2026-11-08 (partial): `find_unresolved_callees` + tests exist; same catalog gap.

Steps: *to be written*

Done when: *to be written*

M6-09 — Idempotence with ruff line-wrapping

- **Status:** open
- **Priority:** P1
- **Area:** Testing / QA
- **Readiness:** needs recipe

Context: verified on corpus.

Audit 2026-11-08 (partial): generic idempotence suites green (incl. ruff format round-trips), but no kwargs-specific corpus assertion that `calls.add-keywords` fires; typing-only catalog used in runner corpus tests.

Steps: *to be written*

Done when: *to be written*

M6-04 — Style controls

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** needs recipe

Context: — `min_positional`, per-callee allow/deny lists (e.g. don't keyword `len(x)` / `print(x)` first arg).

Audit 2026-11-08: still not done — no `min_positional` or per-callee allow/deny anywhere; `FixSettings` /config/catalog params have no kwargs knobs; every keyword-capable positional is converted.

Steps: *to be written*

Done when: *to be written*

Completed

ID	Task	Priority	Area
M6-01	Danger matrix → ~60 tests	P1	Testing / QA
M6-03	Library keyword table	P1	Backend
M6-10	Class-body rebinding of a method name (audit: 9 strict-xfail rows)	P0	Backend
M6-11	Constructor hooks (audit: 6 strict-xfail rows)	P0	Backend
M6-12	Dynamic rebinding markers (audit: 4 strict-xfail rows)	P0	Backend
M6-13	`__getattribute__` interception (audit: 1 strict-xfail row, `getattribute-override`)	P1	Backend

7. Speed and shipping

Scope: Parallelism, reporters, config, packaging, editor integration.
ID prefixes: M7

M7-01 — Kill superlinear costs

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: — semantic worklist, single subject pass bucketed by kind, per-node binding caches.

Audit 2026-11-07 (partial): the largest superlinear cost is removed — `Analyzer.refresh()` re-fingerprinted the whole tree per query (~1 053 walks ≈ 45 s at 12k);

internal queries now use a refresh-free solver path in `semantic/analyzer.py` and `_fingerprint_tree` itself is cheaper. A fresh 3k `cProfile` (post M8-19/20) puts remaining cost in `hints.index` ≈ 17.6 s cum, `provable.scope_types` 1 240 calls ≈ 7.4 s, `resolve.get` 1 755 calls ≈ 6.6 s, ~12M `iter_child_nodes` walks and ~96 fingerprints ≈ 4.7 s — the listed worklist/bucketing/binding-cache work lives in `rules/hints.py`, `rules/provable.py`, `rules/resolve.py`, `rules/engine.py` and `ast.py`'s `semantic` property (main-agent line). Progress 2026-11-08 (agent: claude): a profile of `statistics.py` showed half the run was the docstring-quote probe (`docstring.token_text` per docstring rebuilds the token index each call); `triple_quoted_owners` now reads the module text once. Cold `fix_source`: 11.3 s → 4.0 s (`statistics`), 2.3 s (`os`). Docstring fixes no longer conflict with edits to the same definition (`Constants.docstring_prefix` in `rules/runner.py`), so modules converge in 2 rounds instead of 3. Rule of thumb recorded: never read per-node `token_text` in a loop. Remaining profile: the inference pipeline (`hints._candidates` ≈ 1/3), `hints.index` and `engine._holds`. Progress 2026-11-08 (agent: devin, Q-31 measurement): at 50k lines the cliff is far steeper than the ~n^1.4 seen at 12k — `check` alone exceeded 30 min on the 50,763-line `profile_perf` workload (> 85× the 12k `check`), and `fix_source` multiplies it via `max_rounds=3` plus `_degrade` halving re-running `_attempt` / `_rematch` per split. `cProfile` isolates `hints.index → _candidates → CommentedModule.convert → _graph/backend/checkpoint._infer` (whole-module graph conversion once per hint-index build). Filed as Q-68; the ~50k row in `docs/benchmarks.md` records the lower bound.

Steps: *to be written*

Done when: *to be written*

M7-02 — Perf budgets

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: ≤5 s/12k lines, ≤30 s/100k lines single-threaded.

Audit 2026-11-07 (partial): 12k-line `check` 62→**29.7 s**, `fix` 194→**72.3 s**, total 283→**102 s** with identical findings (2 067; applied 413–432 across concurrent rule work); `docs/benchmarks.md` records the runs. Budget still unmet — remaining cuts sit in `rules/` engine code owned by the M2/M3 line. Progress 2026-11-08 (agent: claude): regression guards added in `tests/test_perf_budgets.py` — a structural counter (token-index rebuilds per fix ≤ 20; was 238 before the `token_text` fix, now 6–8) on three corpus modules, a 3 s clock on a 120-line module, and a slow-tier 15 s clock on the 1 800-line `statistics` (now ≈ 4 s). Second pass 2026-11-08: `hints._call_sites` evaluated `scope_types(...)` eagerly inside `envs.setdefault(...)` for every call; it is now computed once per scope per pass (`_scope_env`), and `_type_rebound_locals` is skipped for functions with no name bound twice. Measured with `benchmarks.profile_perf`: 3k lines check 4.2→2.6 s / fix 7.2→5.2 s; **12k lines check 29.7→21.4 s / fix 72.3→36.9 s**. Budget (≤ 5 s / 12k) **not met** and cannot be met by local tuning: ~50 % of the remaining time is the inference engine (`_graph/backend/engine.py`, ~4 M `ast.walk` steps from thousands of per-node `walk` calls) and growth is ~n^1.4. Remaining plan, in order: (1) precompute parent/descendant indexes once per function in `_graph/backend/engine.py` and replace the per-node `walk` calls; (2) run `check` and `fix` rounds on changed functions only (incremental, reuse of unchanged `HintIndex` parts); (3) M7-04 process-pool across files; (4) re-measure on 12k/100k. Keep M7-02 open until (4).

Steps: *to be written*

Done when: *to be written*

M7-16 — Wire the built-but-unreachable product surface (audit finding)

- **Status:** open
- **Priority:** P1
- **Area:** DevOps / Release
- **Readiness:** ready

Context: `cli/config.py`, `cli/ignores.py`, `cli/baseline.py`, `cli/sarif.py`, `cli/junit.py`, `cli/github.py` are implemented and tested (`tests/test_cli_config.py`, `tests/test_cli_reports.py`) but `astart.__main__` / `cli/commands.py` do not import any of them: `astart check --help` still offers only `--format {text,json}`, `--fail-on`, `--stubs`, and `[tool.astart]` in `pyproject.toml` has no effect. M7-07 is checked on loader evidence; its user-visible acceptance lives here.

Acceptance: `astart check` discovers `pyproject.toml`, applies `select` / `ignore` /overrides/baseline, and emits `--format sarif|junit|github`; `astart fix` honours `safety-ceiling`, `min-confidence`, `checker`, `exclude`; each has a CLI test through `main(argv=…)`; README documents the flags.

M7-04 — Process-pool parallelism

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** needs recipe

Steps: *to be written*

Done when: *to be written*

M7-05 — Streaming `--jobs` CLI

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** needs recipe

Steps: *to be written*

Done when: *to be written*

M7-06 — Memory budget tests

- **Status:** open
- **Priority:** P2
- **Area:** Testing / QA
- **Readiness:** needs recipe

Steps: *to be written*

Done when: *to be written*

M7-08 — Baselines + external per-path ignores

- **Status:** open
- **Priority:** P2
- **Area:** DevOps / Release
- **Readiness:** needs recipe

Context: (suppression comments stay forbidden).

Audit 2026-11-07 (partial): `cli/baseline.py` and `cli/ignores.py` exist with tests in `tests/test_cli_reports.py`; not reachable from the CLI (→ M7-16).

Steps: *to be written*

Done when: *to be written*

M7-09 — Reporters

- **Status:** open
- **Priority:** P2
- **Area:** DevOps / Release
- **Readiness:** needs recipe

Context: SARIF/GitHub-annotations/JUnit/ `--statistics`.

Audit 2026-11-07 (partial): `cli/sarif.py`, `cli/junit.py`, `cli/github.py` exist with tests; `--statistics` does not; none reachable from `astart check` (→ M7-16).

Steps: *to be written*

Done when: *to be written*

M7-10 — LSP diagnostics + code actions

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** needs recipe

Steps: *to be written*

Done when: *to be written*

M7-11 — Pre-commit/CI recipe

- **Status:** open
- **Priority:** P2
- **Area:** DevOps / Release
- **Readiness:** needs recipe

Steps: *to be written*

Done when: *to be written*

M7-12 — Interactive review mode

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** needs recipe

Context: — evidence ladder per guarded fix, "accept all high-confidence".

Steps: *to be written*

Done when: *to be written*

M7-13 — `astart explain <rule> <location>`

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** needs recipe

Context: command.

Steps: *to be written*

Done when: *to be written*

M7-14 — `--backup` /undo journal

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** needs recipe

Steps: *to be written*

Done when: *to be written*

M7-15 — Distribution packaging

- **Status:** open
- **Priority:** P3
- **Area:** DevOps / Release
- **Readiness:** needs recipe

Context: (wheel/entry points) once the product surface stabilizes.

Steps: *to be written*

Done when: *to be written*

Completed

ID	Task	Priority	Area
M7-03	Incremental content-hash cache	P1	Backend
M7-07	`[tool.astart]` pyproject config	P1	DevOps / Release

8. Ontology and inference core

Scope: Rule catalog, predicates, the `_graph` inference engine.
ID prefixes: M8

▶ Background — Spec-catalog remainder (merged from the retired 2026-11-06/07 spec and

M8-21 — M8 exit gate

- **Status:** open
- **Priority:** P0
- **Area:** Testing / QA

- **Readiness:** needs recipe

Context: `verify_catalog` all invariants green; full spec-§5 rule set present; parity vs Exactus KWARGS_TYPEHINTS categories complete.

Steps: *to be written*

Done when: *to be written*

M8-01 — `imports.bare-import` IMP001

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: — `import ast` → `from ast import X` + use-site rewrite, proven when collision-safe.

Audit 2026-11-07 (partial): detector `rules/imports_bare.py` + tests; no catalog rule, no fixer — fixer spec is M8-27, registration is M8-22.

Steps: *to be written*

Done when: *to be written*

M8-02 — `typing.missing-variable-annotation` TYP005

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Audit 2026-11-07 (partial): detector `rules/variable_annotations.py` flags unannotated `Assign` name-targets whose value infers a stable non-`Unknown` type; refuses annotated/private/dunder names, attribute and unpacking targets; tests in `tests/test_rules_type_checks.py` ; not in the catalog (→ M8-22).

Steps: *to be written*

Done when: *to be written*

M8-03 — `typing.no-object-annotation` TYP002

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Audit 2026-11-07 (partial): detector `rules/object_annotations.py` + tests; not in the catalog (→ M8-22).

Steps: *to be written*

Done when: *to be written*

M8-04 — `suppression.marker-comment` SUP001

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Audit 2026-11-07 (partial): detector `rules/suppressions.py` + tests; not in the catalog (→ M8-22).

Steps: *to be written*

Done when: *to be written*

M8-05 — `parsing.module-does-not-parse` PRS001

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Audit 2026-11-07 (partial): detector `rules/parse_module.py` + tests; not in the catalog (→ M8-22). Note `astart check` already reports a syntax-error file as `error:` with exit 2.

Steps: *to be written*

Done when: *to be written*

M8-06 — `typing.return-type-mismatch` / `incompatible-default`

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Audit 2026-11-07 (partial): detectors `rules/type_mismatch.py` (`find_return_mismatches`, `find_incompatible_defaults`) refuse on any indeterminate inference — only a definitive `compatible is False` verdict yields a candidate (bare `Constant` literals are mapped directly since the solver does not index them); tests in `tests/test_rules_type_checks.py`; not in the catalog (→ M8-22).

Steps: *to be written*

Done when: *to be written*

M8-07 — Docs family DOC001/003/004/006

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: + guarded doc fixers. Progress 2026-11-07: DOC002 (add stub entries), DOC003 (remove stale entries) and DOC007 (tidy whitespace/indent) are catalog rules with proven fixers (`rules/docs_fixes.py`, `rules/docstring_edit.py`); DOC001 (stub pre-pass, `--stub-docstrings`) is done; DOC004/DOC006 stubs remain → Q-61. Keep open until they land.

Audit 2026-11-08 (partial): DOC001/002/003/007 are live catalog rules with wired fixers (`docs.add-parameter-entries`, `docs.remove-stale-entries`, `docs.tidy-docstring` in `Appliers`); DOC001's stub pre-pass runs under `--stub-docstrings` in `runner`. Corpus idempotence green again after the `triple_quoted_owners` pre-computation and `entry_names` dedup fixes (79/79). DOC004 (missing returns) and DOC006 (missing raises) detectors still have no catalog rule.

Audit 2026-11-07 (partial): detectors `rules/docs_missing.py` (DOC001), `rules/docs_stale_params.py` (DOC003), `rules/docs_missing_returns.py` (DOC004), `rules/docs_missing_raises.py` (DOC006) on structured `DocstringNode.entries` + tests in `tests/test_rules_docs_whitespace.py`; no fixers, not in the catalog (→ M8-22).

Steps: *to be written*

Done when: *to be written*

M8-11 — D6 remainder: `Exists` evaluation

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: — predicate semantics implemented; catalog-validation guard removed.

Steps: *to be written*

Done when: *to be written*

M8-12 — D8: procedural rules

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: implemented.

Audit 2026-11-07 (partial): `rules/procedural.py` defines `Candidate` and `span_of`; `Registries.procedures` is still empty and `engine.check` has no path that calls a procedure (→ M8-22).

Steps: *to be written*

Done when: *to be written*

M8-22 — Register the standalone detectors as catalog rules (audit finding)

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** ready

Context: `imports_bare.py` (IMP001), `object_annotations.py` (TYP002), `suppressions.py` (SUP001) and `parse_module.py` (PRS001) have detectors and tests (`tests/test_rules_procedural.py`), but the default catalog still has 5 rules / 7 fixes and `Registries.procedures` is empty, so `astart check` never reports them (also blocks M8-12).

Audit 2026-11-08: catalog is now 7 rules / 10 fixes across 5 JSON files (`typing` , `attributes` , `calls` , `structure` , `docs`), all `owner: astart` , `maturity: stable` , `procedure: null` , and every file's `profiles` / `relations` lists are empty. Code collision to resolve: catalog `attributes.json` already spends `TYP005` on `typing.missing-attribute-annotation` , while M8-02's variable- annotation detector also targets TYP005 — registration must renumber one. Also `variable_annotations.py` , `type_mismatch.py` (return/default mismatch, M8-06) and `call_problems.py` (M6-05…08) detectors await the same path.

Acceptance: the four rules appear in `load_default_catalog` with examples, `verify_catalog` stays green, `check` reports them with spans and fingerprints, IMP001 offers a proven fix only when collision-safe.

M8-27 — IMP001 fixer: bare `import a` → `from a import b` + use-site rewrite

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** ready

Context: Convert module-level bare imports whose every use is an attribute **load** into per-attribute `from` -imports, decided independently per attribute; keep `import a` for any site that cannot convert.

Scope: `rules/imports_bare.py` evidence → a new applier/fix entity; module-level `import a` and dotted `import a.b.c` only (no aliases, no conditional/ `try` / `TYPE_CHECKING` imports, no function-level imports, no star imports). Rewrite `a.<attr>` loads to the bound name; for `import a.b.c` , `a.b.c.<attr>` → `from a.b.c import <attr>` . Chains convert one level: `a.b.c` → `from a import b` + `b.c` . Acceptance — must convert: `import a` + `print(a.b)` → `from a import b` + `print(b)` . Acceptance — must refuse or partially keep `import a` : (a) attribute stores/dels/aug-assign/for- and with-targets (`a.b = 3` , `del a.b` , `a.b += 1`) — `b = 3` would rebind locally, not set `a.b` ; the store site keeps `import a` (with only stores present, no `from` import is emitted at all); (b) name collisions — `def b()` , `b = …` , `import b` , `from x import b` , or a parameter/ `global` binding of `b` in scope → skip that attribute; (c) existing `from a import b [as c]` — reuse the already-bound name (`a.b` → `c` , no duplicate import) even when an unrelated local `b` exists, since `c` is the correct target; (d) bare- `a` uses (`f(a)` , `x = a` , `a.b` inside string annotations or other unrewritable positions) — `import a` survives for those sites; (e) `a.b` loads inside nested functions/lambdas still convert (scope of the binding is unchanged); (f) conditional or `try` -guarded imports, `import a as x` , multi-name `import a, os` , and modules whose attribute may come from `__getattr__` /lazy submodule import are out of scope for `proven` — emit at `guarded` at best and let the external checker veto.

Audit 2026-11-07 (partial): fixer module `rules/imports_bare_fix.py` (`rewrite_bare_imports`) implements the per-attribute plan — load-only rewrite points, store/del/augassign/for-target veto per attribute, collision and existing- `from` -alias reuse, bare-use and dotted-prefix keeps, nested function/lambda scopes; 19 danger-matrix tests in `tests/test_rules_imports_bare_fix.py` green incl. idempotence.

Evidence: danger-matrix tests in the style of `tests/test_rules_kwargs_danger.py` covering every refusal above plus alias/ `as` -import, dotted-path, chain and multi-attribute cases; corpus idempotence (fix-fix byte-equal) and the writer fuzzer's compile/comment/strip-equality invariants apply to rewritten output. Remaining: applier/catalog registration (→ M8-22) and corpus/fuzzer runs over rewritten output.

M8-08 — Whitespace/blank-line normalisation

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** needs recipe

Context: rules.

Audit 2026-11-07 (partial): detectors `rules/whitespace_rules.py` (blank-line runs, trailing space) + tests; not in the catalog (→ M8-22).

Steps: *to be written*

Done when: *to be written*

M8-09 — `Constants` -class grouping with `Final`

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** needs recipe

Context: rule.

Audit 2026-11-07 (partial): detector `rules/constants_grouping.py` flags loose `UPPER_SNAKE` module constants outside a `Constants` class + tests; no fixer, not in the catalog (→ M8-22).

Steps: *to be written*

Done when: *to be written*

M8-10 — Delegation relation to ruff

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** needs recipe

Context: documented and wired.

Audit 2026-11-07 (partial): delegation contract documented in `docs/rule-authoring.md` (`owner: "ruff"` + `delegates_to` relation, FMT001/ `format.line-length` as the planned entry); schema already supports both fields; the catalog entry itself is M8-22 scope.

Steps: *to be written*

Done when: *to be written*

M8-13 — Profiles + relations

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** needs recipe

Context: (`supersedes` / `conflicts_with` / `implies`) exercised in the default catalog.

Steps: *to be written*

Done when: *to be written*

M8-28 — String literal normalisation fixer

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** ready

Context: Two value-preserving rewrites of plain `str` constants: (a) literals containing `\n` escapes become multiline triple-quoted strings with real newlines; (b) literals whose source line exceeds the line-length budget become parenthesised implicit concatenations split at safe boundaries.

Scope: `Constant` `str` nodes only — no f-strings/ `JoinedStr` , no `bytes` , no raw strings for (a) (an `r"…\n…"` is a literal backslash, not a newline), no strings already triple-quoted or already implicit concatenations. (b) applies to `str` values in assignment values, call arguments, `return` / `yield` values and parenthesised sub-expressions; the emitted form is `("part one" "part two")` — wrapping parens required so a bare split cannot be mistaken for two statements. Acceptance — must convert: `a = "line one\nline two"` → `a = """line one` ↵ `line two"""` (escape sequences other than `\n` may stay escaped inside the triple-quoted form); `a = "<80+ chars>"` → `a = ("<chunk>" "<chunk>")` with every emitted line inside the budget; the same split inside a call `f("<long>")` → `f(("<chunk>" "<chunk>"))` -equivalent argument form. Acceptance — must refuse: (a) strings containing `"""` , a trailing `\` , or content that would need escaping changes to stay valid (quote style may flip `"` ↔ `'` only when it reduces escapes, never adds them); (b) docstrings — module/class/function docstring positions are statement-sensitive; converting them is allowed only when the triple-quoted form keeps the same first-statement position and the project's docstring style permits it, otherwise skip; (c) split points inside escape sequences (`\n` , `\x41` , `\u1234` , `\N{…}`), inside a word, or inside an existing implicit concatenation — split only at whitespace boundaries, prefer spaces, never inside an escape; (d) strings already fitting the budget, empty strings, strings that are dict keys/ `__all__` entries/ `match` patterns where reformatting buys nothing (style choice documented, not correctness); (e) f-strings, bytes, raw strings for (a), and any literal inside a larger expression where added parens could rebind precedence (e.g. `x + "long"` , `"a" "b" + c`) — wrap or refuse, never emit invalid or re-precedenced Python.

Audit 2026-11-07 (partial): fixer module `rules/strings_normalize.py` (`normalize_strings`) - token-exact segment rewrites on eligible `Constant str` nodes; `\n` -escape to triple-quote with real newlines (refuses triple-quote content, trailing backslash), overlong literals to parenthesised implicit concatenation at word boundaries with the boundary space preserved inside quotes; refuses f-strings, bytes, raw/triple-quoted, docstrings, dict keys, `__all__` members, match patterns; emitted lines respect the budget; 15 tests in `tests/test_rules_strings_normalize.py` green incl. value round-trip and idempotence. Remaining: applier/catalog registration (M8-22) and writer-fuzzer runs.

Evidence: danger-matrix tests mirroring `tests/test_rules_kwargs_danger.py` for every refusal above; `ast.parse` validity + value equality

(`ast.literal_eval` round-trip) on every rewritten literal; idempotence (second pass is a no-op) and the writer fuzzer's compile/comment/strip-equality invariants.

M8-29 — F-string interior normalisation fixer

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** ready

Context: Canonicalise the *inside* of `JoinedStr` replacement fields without touching what they evaluate to: `f"{ x }"` → `f"{x}"`, `f"{x !r}"` → `f"{x!r}"`, `f"{ x !s :>8 }"` → `f"{x!s:>8}"`.

Scope: `JoinedStr` / `FormattedValue` nodes only — whitespace and conversion-flag *placement* normalised; the expression text, the conversion (`!s` / `!r` / `!a` — semantics differ, never added/removed/ reordered) and the format spec are preserved verbatim, including whitespace inside the spec where it is significant. Acceptance — must convert: `f"{ x }"` → `f"{x}"`; `f"{x !r}"` → `f"{x!r}"`; `f"{x:>8 }"` keeps its spec untouched while outer spacing is fixed; nested replacement fields inside specs (`f"{x:{w}.{p}f}"`) keep their own interior normalised recursively; quote style inside the f-string may flip only when it removes escapes (Python ≥3.12 same-quote nesting; below that the outer/inner quotes must differ — emit the minimal-escape form for the running `target-version`). Acceptance — must refuse: (a) any rewrite that would change evaluated output — format-spec whitespace is significant (`f"{x: >5}"` ≠ `f"{x:>5}"` for some types), `=` -debug text (`f"{x=}"`) is output, `!r` / `!s` / `!a` choose different `__str__` / `__repr__` / `__ascii__` paths; (b) f-strings containing `\n` real newlines or `"""` content that cannot stay inside the chosen quote style — hand to M8-28 or skip; (c) expressions with side-effecting or order-sensitive text the fixer cannot reproduce byte-faithfully (walrus targets, starred args, unusual spacing inside the expression itself is *not* normalised — only the field delimiters are); (d) conversion of whole f-strings to `.format()` /concatenation — out of scope; this task is interior formatting only.

Evidence: danger-matrix tests for every refusal; output-equality proof per case (`eval` / `ast.literal_eval` where legal, or a stub object recording `__format__` / `__str__` calls for spec cases); idempotence; writer-fuzzer invariants on rewritten modules.

Audit 2026-11-07 (partial): fixer module `rules/fstring_normalize.py` (`normalize_fstrings`) - tokenize-driven field-interior rewrites; trims edge whitespace around expr/`!conv`/`:` separators, preserves expression interior spacing, conversion flags and format-spec text verbatim, recurses into nested spec fields; refuses `=` -debug fields, unbalanced brackets and non-f-string braces; 11 tests in `tests/test_rules_fstring_normalize.py` green incl. idempotence.

Note: spec text is preserved byte-for-byte, so `f"{x !s :>8 }"` yields `f"{x!s:>8 }"` (spec trailing space kept) rather than the spec example's `:>8}`-stripped form - chosen because format-spec whitespace is significant. Remaining: applier/catalog registration (M8-22) and writer-fuzzer runs.

M8-30 — Multiline-string indent normalisation + `textwrap.dedent` fixer

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** ready

Context: Two related rewrites: (a) docstrings are normalised to house style — summary placement, consistent continuation indentation, closing quotes on their own line — preserving `inspect.cleandoc` semantics; (b) non-docstring multiline `str` literals whose continuation lines share a common indent are offered `textwrap.dedent(...)` wrapping (with `ensure_names` import insertion) *or* literal re-indentation, chosen by evidence.

Scope: `Constant` `str` nodes spanning ≥2 lines with a non-empty common leading-whitespace prefix on continuation lines; docstrings via the structured `DocstringNode` entries, not raw text. Acceptance — must convert: module/class/function docstrings violating house indent style → normalised form (`cleandoc`-equivalent value); `x = """\n body\n """` where downstream evidence (passed to `textwrap.dedent` / `.strip()`-family, compared against dedented constants, or the module's own convention) shows the *intended* value is the dedented text → `x = textwrap.dedent("""…""")`; strings where only the literal's layout is wrong (misplaced closing quotes, ragged continuation indent with no common prefix) → re-indented literal, value byte-identical. Acceptance — must refuse: (a) `dedent` wrapping when the value change is unproven — the runtime value of `textwrap.dedent(literal)` differs from the literal whenever the first content line or whitespace-only lines carry indent; emit at `suggest` / `guarded` only with recorded evidence, never `proven` on style alone; (b) mixed tabs/spaces, first-line content carrying indent (`"""text\n more"""` — dedent cannot touch line 1), strings already inside `dedent` / `inspect.cleandoc` / `\.strip()` chains, f-strings/ `JoinedStr` and `bytes`; (c) literals whose common prefix is empty or whose "indent" is meaningful content (ASCII-art, code samples, aligned columns) — detect via evidence slots, not heuristics alone; (d) docstrings in stub/ `...` bodies and `__future__`-adjacent positions where statement order is load-bearing.

Evidence: danger-matrix tests; value-equality proof — `ast.literal_eval` for re-indented literals and `textwrap.dedent(literal_eval(text))` equality for wrapped ones; `inspect.getdoc` equivalence for docstring cases; idempotence; writer-fuzzer invariants.

M8-31 — `suppression.lint-ignore-config` SUP002

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** needs recipe

Context: (Exactus alias CMP004, cost `project`, no fix). Flags project configuration that silences lint checks wholesale (file-level or per-file ignore configuration), so the finding names where coverage is being waived rather than flagging the waivers themselves.

Steps: *to be written*

Done when: *to be written*

M8-32 — `suppression.type-check-config` SUP003

- **Status:** open
- **Priority:** P2

- **Area:** Backend
- **Readiness:** needs recipe

Context: (Exactus alias CMP005, cost `project`, no fix). Flags type-checker configuration that disables or weakens checking (e.g. pyright/basedpyright `typeCheckingMode` off, mypy `ignore_errors`), project scope.

Steps: *to be written*

Done when: *to be written*

M8-33 — `config.pyproject-missing` CFG001

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** needs recipe

Context: (Exactus alias CMP006, cost `project`, fix safety `suggest`). Reports a project with no `pyproject.toml`; the fix only suggests creating a minimal one, never auto-applies.

Steps: *to be written*

Done when: *to be written*

M8-34 — Method/classifier quartet OOP001–OOP004

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** needs recipe

Context: (Exactus aliases PY004–PY007, cost `scope`, fixes `guarded`): `oop.method-should-be-classmethod`, `oop.method-should-be-static`, `oop.classmethod-should-be-static`, `oop.staticmethod-uses-class`. Needs `MethodIndex` / `receiver_of` evidence plus "uses `self` / `cls`" facts per method.

Steps: *to be written*

Done when: *to be written*

M8-35 — `naming.exception-message-variable` NAM001

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** needs recipe

Context: (Exactus alias PY003, cost `scope`, fix `proven` when control flow is safe): `msg = "..."` immediately before `raise X(msg)` folds into `raise X("...")`; refuse when `msg` is read elsewhere or rebound.

Steps: *to be written*

Done when: *to be written*

M8-36 — Simplification family SIM001–SIM006

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** needs recipe

Context: (Exactus aliases PYS001–PYS006, fixes `proven`): `simplification.inline-return`, `walrus-in-condition`, `inline-single-use`, `dead-constant-store`, `inline-in-return`, `inline-via-conditional`. Exactus's skip reasons become the fixes' `preconditions` (evaluation-order safety), per ontology spec §5.

Steps: *to be written*

Done when: *to be written*

M8-37 — Docs family remainder DOC005, DOC007–DOC013

- **Status:** open
- **Priority:** P2

- **Area:** Backend
- **Readiness:** needs recipe

Context: plus the doc fixers the ontology spec attaches to the docs rules: detectors `docs.return-yield-mismatch` (DOC005), `docs.missing-attribute` (DOC007), `docs.malformed-docstring` (DOC008), `docs.analysis-failure` (DOC009), `docs.missing-description` (DOC010), `docs.stale-raises` (DOC011), `docs.stale-attribute` (DOC012), and `docs.format-style` (DOC013 — normalise docstring content to Google style, no Exactus alias). Fixes: missing-section rules offer `set_docstring` with a generated entry (`guarded` , confidence-gated); stale-section rules offer removal (`suggest`).

Steps: *to be written*

Done when: *to be written*

M8-14 — `Maturity` lifecycle → defaults

- **Status:** open
- **Priority:** P3
- **Area:** Backend
- **Readiness:** needs recipe

Steps: *to be written*

Done when: *to be written*

M8-38 — `structure.shebang` SHB001

- **Status:** open
- **Priority:** P3
- **Area:** Backend
- **Readiness:** needs recipe

Context: (fix `guarded`): missing, malformed or wrong-interpreter shebang lines on executable scripts; rewrite only with recorded evidence.

Steps: *to be written*

Done when: *to be written*

M8-39 — CLI watch mode

- **Status:** open
- **Priority:** P3
- **Area:** DevOps / Release
- **Readiness:** needs recipe

Context: The only out-of-scope item left from the check/fix CLI plan (`--select` / `--ignore` , config files and parallelism already landed or are tracked under M7-04/M7-05/M7-07): a `astart check --watch` / `fix --watch` mode that re-runs on file changes.

Steps: *to be written*

Done when: *to be written*

Completed

ID	Task	Priority	Area
M8-23	New `Final` constants join an existing `Constants` class (requested 2026-11-07)	P1	Backend
M8-24	Writer replaces elements of a child list with atoms (needed by M8-23)	P1	Backend
M8-15	Rule authoring kit	P3	Documentation
M8-16	Fix verified wrong types	P1	Backend
M8-17	`list(d)` → items-not-keys	P1	Backend
M8-18	`seq[0]` → `Sequence[int]` over-broad	P1	Backend
M8-19	Imported-constructor argument typing	P1	Backend
M8-20	`self.` receiver typing in the analyzer	P1	Backend

9. Cross-cutting engineering

Scope: Calibration, determinism, error containment.
ID prefixes: CC, EV

CC-01 — Confidence calibration

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: — numeric confidence per candidate from evidence level, calibrated on benchmarks (pairs with M1-15).

Steps: *to be written*

Done when: *to be written*

CC-02 — Determinism audit

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: — stable ordering/names, protocol name lockfile, no dict/set-order or time dependence anywhere.

Steps: *to be written*

Done when: *to be written*

CC-03 — Error containment v2

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: — per-file timeout + per-fix watchdog on top of existing per-file/per-fix isolation.

Steps: *to be written*

Done when: *to be written*

CC-04 — Property + mutation testing

- **Status:** open
- **Priority:** P1
- **Area:** Testing / QA
- **Readiness:** needs recipe

Context: — grammar-random programs → strip → fix → check; mutation testing on policy code; regression test per reviewer finding.

Steps: *to be written*

Done when: *to be written*

CC-06 — Convention guard

- **Status:** open
- **Priority:** P1
- **Area:** Testing / QA
- **Readiness:** needs recipe

Context: — no suppression comments, no `Any` / `object`, keyword-only calls, Google docstrings, `@final Constants` — keep `test_rules_review_fixes` family green as rules evolve.

Steps: *to be written*

Done when: *to be written*

CC-07 — Commit hygiene (audit finding)

- **Status:** open
- **Priority:** P1
- **Area:** DevOps / Release
- **Readiness:** ready

Context: 148 paths differ from the last commit (`120272c`) — about 35 source/test files and ~100 new corpus modules — including two sessions' work. Nothing from M0–M2 is committed, so no milestone diff is reviewable or revertable.

Acceptance: one commit per finished milestone slice (M0, M1-so-far, M2-so- far, corpus, parallel-session reporters/config) on a branch; the suite and `ruff check src tests benchmarks scripts` green at each commit.

Completed

ID	Task	Priority	Area
EV-01	Exactus gate evidence, 2026-11-07	P0	Testing / QA
CC-05	User-facing evidence ladder + refusal catalogue	P2	Documentation

10. Contract negotiation

Scope: Two-way reconciliation of types (the ant farm).
ID prefixes: F

▶ Background — 18.12 Contract negotiation — two-way reconciliation (the "ant farm")

F-17 — Seeds from existing evidence, origin-tagged: default literal (`default`), proven…

- **Status:** open
- **Priority:** P0
- **Area:** Backend
- **Readiness:** needs recipe

Context: Seeds from existing evidence, origin-tagged: default literal (`default`), proven call-site types (`call_sites`), syntax proofs (`syntax`), declarations (`declared`), engine candidates (`inferred`). Progress 2026-11-08: syntax, declared, call-site and default seeds are built; engine proposals are *corroboration* only (they are compared with the settled type in `policy.accept` , never unioned into a belief, because `int | str` from the engine would silently widen call-site evidence).

Steps: *to be written*

Done when: *to be written*

F-18 — Acceptance policy `negotiation/policy.py` : settled type + independent origin + no open…

- **Status:** open
- **Priority:** P0
- **Area:** Backend
- **Readiness:** ready

Context: Acceptance policy `negotiation/policy.py` : settled type + independent origin + no open conflict + spellable (known names, no `object` / `Any`) → `Hint` with new `HintBasis.negotiated` ; confidence by origin mix in `confidence.py` ; new `Rejection` values (`negotiation conflict` , `negotiation ambiguous`) with `docs/skip-reasons.md` and refusal-catalogue rows (existing doc tests must stay green). Progress 2026-11-08: `negotiation/policy.py` — every member of the settled type needs a proof (syntax/declared), or two different soft signals (call sites + default), or one soft signal plus engine agreement; opacity spreads downstream; `object` / `Any` and a parameter typed `None` are refused. Open: `HintBasis.negotiated` , confidence table, the two new `Rejection` values and their docs. Done in part 2026-11-08: `HintBasis.negotiated` (confidence 0.95), `typing.add-variable-annotation` (0.95). The new `Rejection` values and their docs are still open.

F-20 — Behind `FixSettings.negotiate` (CLI `--negotiate`), let the negotiator replace the…

- **Status:** open
- **Priority:** P0
- **Area:** Backend
- **Readiness:** needs recipe

Context: Behind `FixSettings.negotiate` (CLI `--negotiate`), let the negotiator replace the 12-pass propagation fixpoint in `HintIndex.judge_parameters` ; first goal is *identical* output on the gold set, then a strict superset with 0 new contradictions. Progress 2026-11-08: negotiation now *supplements* the hint index instead of replacing the 12-pass loop (`HintIndex.supplement` , `LazyHints.supplemented` , `FixSettings.negotiate` , default **on**, CLI `--no-negotiate`): a negotiated hint is used only for nodes the hint index has none for, and a parameter that would get a structural `Protocol` keeps the Protocol (`own_hint_for`). Gold: 561 cases, negotiation changes 0 written annotations and adds 24 reviewed ones (`test_gold_negotiation_is_a_reviewed_superset`). Replacing the loop outright stays open until the shadow gap (F-37) is closed.

Steps: *to be written*

Done when: *to be written*

F-28 — Determinism and idempotence: hash-seed test (as Q-15), `fix` ∘ `fix` byte-equal on both…

- **Status:** open
- **Priority:** P0
- **Area:** Testing / QA
- **Readiness:** needs recipe

Context: Determinism and idempotence: hash-seed test (as Q-15), `fix` ∘ `fix` byte-equal on both corpora with negotiation on, order-independence property test on random graphs.

Steps: *to be written*

Done when: *to be written*

F-29 — Gate: refresh `baseline.json` and `baseline_ext.json` with negotiation on; accept only…

- **Status:** open
- **Priority:** P0
- **Area:** Testing / QA
- **Readiness:** needs recipe

Context: Gate: refresh `baseline.json` and `baseline_ext.json` with negotiation on; accept only precision ≥ 99.5 %, 0 new checker errors, 0 crashes, and an external-corpus recall gain (target ≥ +2 points over 19.3 %). Record per-origin precision.

Steps: *to be written*

Done when: *to be written*

F-07 — Complete the transfer table, forward *and* backward, per relation: key/value (`dict[K,…

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: Complete the transfer table, forward *and* backward, per relation: key/value (`dict[K, V]`), argument ↔ parameter, call*result ↔ return, attribute/instance*field/class*field, operator (numeric tower,* `str % x` *), context*enter/exit (`__enter__` result), awaitable unwrap, generator send/yield/return. A relation with no sound reverse gives `None` (no message), never a weaker guess.

Contract builders (AST → contracts; reuse `CallBindings` / `MethodIndex` so binding trust is shared with the keyword fixer):

Steps: *to be written*

Done when: *to be written*

F-13 — Builder: capabilities

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: — `int(x)` , `x.y` , `x()` , `x[k]` , `len(x)` , `for _ in x` , `with x` , `await x` , `x + 1` — as capability sets on the node; feeds `usage_abc` (M4-07) and Protocol synthesis instead of re-deriving usage separately.

Steps: *to be written*

Done when: *to be written*

F-15 — Builder: library calls

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: — declared parameters/results from `LibraryIndex` /typeshed as *declaration* contracts (stdlib first), so `open(path)` gives `path` a `str | PathLike` capability.

Steps: *to be written*

Done when: *to be written*

F-16 — Builder: declared annotations, base-class and override signatures (E1, M2-07) as…

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: Builder: declared annotations, base-class and override signatures (E1, M2-07) as declaration contracts.

Policy and wiring:

Steps: *to be written*

Done when: *to be written*

F-21 — Returns as union contracts: join of all exits subsumes `_join_types`; composes with…

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: Returns as union contracts: join of all exits subsumes `_join_types`; composes with `_flow_returns`; union returns need the same independence rule as parameters (M4-11 return half).

Steps: *to be written*

Done when: *to be written*

F-22 — Backward reconciliation of parameters: use-site capabilities/narrowing intersect the…

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: Backward reconciliation of parameters: use-site capabilities/narrowing intersect the call-site union (sites `int` and `str` but body does `x + 1` → report a conflict, do not write `int | str`).

Steps: *to be written*

Done when: *to be written*

F-23 — Conflict reporting: a detection rule (`typing.type-conflict`, info) that lists the…

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: Conflict reporting: a detection rule (`typing.type-conflict`, info) that lists the entity, both types and the explanation trail; latent bugs surface here instead of being silently refused.

Steps: *to be written*

Done when: *to be written*

F-24 — Calibrate the lossy collapses with the X-05 method (≥ 97 % exact, 0 contradictions, n ≥…

- **Status:** open
- **Priority:** P1
- **Area:** Testing / QA
- **Readiness:** needs recipe

Context: Calibrate the lossy collapses with the X-05 method (≥ 97 % exact, 0 contradictions, n ≥ 100, stdlib *and* external corpus): `int | float` → `float`, `list | tuple` → `Sequence`, `list | set` → `Collection`, `dict | defaultdict` → `MutableMapping`. Enable only the categories that pass.

Steps: *to be written*

Done when: *to be written*

F-26 — Explanation trail per candidate (origin, path, relation, distance) kept through merges;…

- **Status:** open

- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: Explanation trail per candidate (origin, path, relation, distance) kept through merges; `astart explain` prints the evidence path of a negotiated annotation (M7-13).

Steps: *to be written*

Done when: *to be written*

F-27 — Performance: graph build linear in nodes; per-function incremental rebuild; budget tests…

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: Performance: graph build linear in nodes; per-function incremental rebuild; budget tests in `tests/test_perf_budgets.py`; shares the precomputed indexes of M7-02.

Steps: *to be written*

Done when: *to be written*

F-30 — `docs/negotiation.md` (model, adaptation rule, how to add a contract builder or a…

- **Status:** open
- **Priority:** P1
- **Area:** Documentation
- **Readiness:** needs recipe

Context: `docs/negotiation.md` (model, adaptation rule, how to add a contract builder or a transfer rule), `docs/evidence-ladder.md` and README Limits updated, architecture page regenerated. Progress 2026-11-08: `docs/negotiation.md` (model, adaptation rule, six soundness rules, how to extend). README flags documented. `docs/evidence-ladder.md` and the architecture page still to update.

Steps: *to be written*

Done when: *to be written*

F-31 — Gold family `negotiated` (≥ 60 cases: cycles, containers, narrowing, conflicts, unions,…

- **Status:** open
- **Priority:** P1
- **Area:** Testing / QA
- **Readiness:** needs recipe

Context: Gold family `negotiated` (≥ 60 cases: cycles, containers, narrowing, conflicts, unions, refusals) in `tests/test_gold_expansion.py`; termination fuzz on random graphs.

Steps: *to be written*

Done when: *to be written*

F-33 — Cross-module edges through `ProjectIndex` (argument/return/attribute contracts across…

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: Cross-module edges through `ProjectIndex` (argument/return/attribute contracts across files; M3-03 evidence enters as `call_sites` seeds).

Steps: *to be written*

Done when: *to be written*

F-34 — Safety bounds: per-module event and wall-clock budget; on overrun fall back to the…

- **Status:** open
- **Priority:** P1
- **Area:** Backend

- **Readiness:** needs recipe

Context: Safety bounds: per-module event and wall-clock budget; on overrun fall back to the existing `HintIndex` path and report it (CC-03).

Steps: *to be written*

Done when: *to be written*

F-37 — Default-literal parameters in the negotiator (the last big shadow gap: 70 annotations, 60…

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: Default-literal parameters in the negotiator (the last big shadow gap: 70 annotations, 60 exact, 3 wrong). The hint index accepts a literal default with the engine's agreement even when some callers are unknown; the negotiator treats unknown callers as opaque. Port the tiered confidences (`confidence.by_literal`) as a per-origin/per-type table (F-18) instead of copying the looser rule.

Steps: *to be written*

Done when: *to be written*

F-40 — Comprehensions as derived nodes (the loop variable typed from the iterable's element…

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: Comprehensions as derived nodes (the loop variable typed from the iterable's element node): 44 variables of the dev pair are blocked as "container literal mutated or not provable", most of them comprehensions and dict displays over parameters.

Steps: *to be written*

Done when: *to be written*

F-41 — Calls on builtin-typed receivers (`value.strip()` , `items.copy()` , `d.get(k)`) as derived…

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: Calls on builtin-typed receivers (`value.strip()` , `items.copy()` , `d.get(k)`) as derived nodes using the typer's method tables, and `list(...)` / `dict(...)` / `sorted(...)` of node arguments.

Steps: *to be written*

Done when: *to be written*

F-43 — Quote instead of refusing: where a negotiated annotation names a class not yet bound (`->…

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: Quote instead of refusing: where a negotiated annotation names a class not yet bound (`-> URIBuilder` inside its own class), write the string form (or `Self` on 3.11+) instead of dropping it.

Steps: *to be written*

Done when: *to be written*

F-25 — Objective choice between a concrete union and an ABC abstraction (coverage, narrowness,…

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** needs recipe

Context: Objective choice between a concrete union and an ABC abstraction (coverage, narrowness, existing alias reuse over a new Protocol, never `Any` / `object`); ties to M5-09/M5-10.

Steps: *to be written*

Done when: *to be written*

F-32 — Retire superseded items: Q-37/Q-38 (slots / site reliability) → F-04/F-02;…

- **Status:** open
- **Priority:** P2
- **Area:** Documentation
- **Readiness:** needs recipe

Context: Retire superseded items: Q-37/Q-38 (slots / site reliability) → F-04/F-02; cross-reference M2-09, M4-07, M4-11, M4-17 to the builders that deliver them.

Steps: *to be written*

Done when: *to be written*

Completed

ID	Task	Priority	Area
F-01	Data model `negotiation/model.py` : `Relation` (assigns, returns, argument, call_result,…	P0	Backend
F-02	Provenance: a candidate carries the set of `Origin` s that support it; `independent` means…	P0	Backend
F-03	Type algebra `negotiation/algebra.py` (`union` , `intersect` , `members`) over type…	P0	Backend
F-04	`negotiation/beliefs.py` `Belief` : add by polarity (union keeps the best confidence per…	P0	Backend
F-05	`negotiation/graph.py` `ContractGraph` : nodes by stable path text, edges with…	P0	Backend
F-06	`negotiation/negotiator.py` `Negotiator.solve` : queue of ants, wormhole memo `(path,…	P0	Backend
F-07a	`negotiation/transfer.py` : first transfer table	P0	Backend
F-08	Builder: assignments (plain, annotated = declaration, augmented, tuple unpacking,…	P1	Backend
F-09	Builder: returns and calls	P1	Backend
F-10	Builder: containers	P1	Backend
F-11	Builder: attributes	P1	Backend
F-12	Builder: narrowing	P1	Backend
F-14	Builder: operators using the typer's tables (numeric tower, str/bytes/list concat,…	P1	Backend
F-35	Protocol-dunder results in the negotiator (`__len__` → `int` , `__exit__` → `bool / None` ,…	P0	Backend
F-36	Calibrated `bool` returns in the negotiator: returns that are boolean by syntax…	P0	Backend
F-38	Lost contributions: a transfer that yields nothing for a union/declaration contract…	P0	Backend
F-39	Opacity reasons: every opaque node records why (`Built.why`), the groundwork for `astart…	P1	Backend
F-42	Names that exist when an annotation is evaluated: a negotiated parameter or return hint…	P0	Backend
F-19	Shadow mode: run the negotiator next to `HintIndex` , record every difference (new hint,…	P0	Testing / QA

11. Untyped to typed conversion

Scope: Converting `dev/untyped.py` into `dev/typed.py` automatically.
ID prefixes: P, T, W

▶ Background — 18.13 Untyped ⇄ typed conversion (the author's fixture pair)

▶ Background — 18.14 Ideas from existing tools (reviewed 2026-11-08)

▶ Background — 19. Ready-to-run work queue (2026-11-08)

▶ Background — 19.2 Coverage work (each task moves a pair number; use the burn-down loop)

▶ Background — 19.3 Rules for writing new tasks here

P-02 — Variable annotations (`x: int = 1`) for module, class and function locals behind…

- **Status:** open
- **Priority:** P0
- **Area:** Backend
- **Readiness:** needs recipe

Context: Variable annotations (`x: int = 1`) for module, class and function locals behind `--annotate-variables` (a profile, off by default: gratuitous locals are noise elsewhere). Nodes already exist for single-binding locals (F-08, F-10); settle them with the negotiation policy and write `name: T = value` through a new applier; the M8-02 detector (`variable_annotations.py`) supplies the rule. Target: the 329 variables of the pair. Progress 2026-11-08: `astart fix --annotate-variables` (also on automatically in modules that already annotate a function-local variable) writes `name: T = value` for single-binding locals — rule TYP007 `typing.missing-variable-annotation` , fix `typing.add-variable-annotation` (guarded, 0.95), settled by `negotiation/variables.py` from proof-origin evidence only. Soundness work it forced: literal containers are typed only while every later use is understood (`append` / `insert` / `add` join the element type, a dict must stay read-only, `*x` / `**x` unpacking is a read); attributes holding a mutable container with an unrecognised use are opaque; wrapper contracts join the elements first (`list[int | str]` , not `list[int] | list[str]`). Dev pair: variables 0 → **53 of 335 exact, 0 wrong** (+4 the author's alias `IntDict`). Open: class-level and module-level variables, attribute annotations from negotiation, comprehension values, calls into libraries (F-15).

Steps: *to be written*

Done when: *to be written*

P-03 — Return types that are "not provable" (127 of the pair's misses): returns through locals,…

- **Status:** open
- **Priority:** P0
- **Area:** Backend
- **Readiness:** needs recipe

Context: Return types that are "not provable" (127 of the pair's misses): returns through locals, containers, attributes and calls to other functions of the module — delivered by F-10…F-14 and F-20; re-measure with the pair after each. Progress 2026-11-08: `MethodIndex` no longer treats a *read* of `globals()` / `vars()` / `locals()` (`globals()["f"](x)` , `.get(...)`) as a rebinding of module names — only stores, `update` , aliases, `exec` and `eval` still do — which unblocked all 152 calls to module functions in `dev/untyped.py` (0 bound before). `--negotiate` (new, off by default) lets negotiated parameter and return hints supplement the hint index where it has none. Dev pair parameters+returns exact 11 → 53 with it; resolver pair 71 → 109. Both baselines unchanged without the flag; gate with `--negotiate` pending.

Steps: *to be written*

Done when: *to be written*

P-08 — The reverse direction, `astart untype` : lossless removal of parameter, return and…

- **Status:** open
- **Priority:** P0
- **Area:** Backend
- **Readiness:** needs recipe

Context: The reverse direction, `astart untype` : lossless removal of parameter, return and variable annotations (and, with `--drop-synthesized` , of Protocols/TypeVars/imports that only the annotations used), keeping comments, layout and docstrings; round trip `untype(typed)` must equal the author's untyped file after `ast.dump` (today `strip_annotations` is benchmark-only and rewrites the text).

Steps: *to be written*

Done when: *to be written*

W-01 — Record the verification commands and their baseline results

- **Status:** open
- **Priority:** P0
- **Area:** Testing / QA
- **Readiness:** ready

Context: Write down the outputs once, then use the commands in every later task. 1. Shadow comparison (~8 min, run in background): `PYTHONPATH=src:. python -m benchmarks.shadow tests/corpus_ext --jobs 8 --json /tmp/shadow.json` then read `/tmp/shadow.json` : the group `only_negotiated` must show `wrong: 0` . 2. External recovery gate (~12 min, background): `PYTHONPATH=src:. python -m benchmarks recovery --corpus tests/corpus_ext --jobs 8 --json /tmp/ext.json` `PYTHONPATH=src:. python -m benchmarks gate --report /tmp/ext.json --baseline benchmarks/baseline_ext.json` must print `PASS` . 3. Stdlib recovery gate (~3 min): same without `--corpus` and without `--baseline` ; must print `PASS` . 4. Semantic equivalence needs 5 packages that are not installed. Install them to a throwaway directory (never into the user's environment): `python -m pip install --no-deps --target /tmp/semdeps toml==0.10.2 tomlkit==0.13.3 boltons==25.0.0 more-itertools==11.2.1` and `python -m pip install --no-deps --no-binary :all: --target /tmp/semdeps2 tomli==2.5.0` . Then in Python: `from benchmarks.semantic import load_projects, run_project` ; for each project,

`run_project(project=p, source=Path("/tmp/semdeps")/p.package)` (`/tmp/semdeps2` for `tomli`); a project passes when `same_imports and same_result` . The other 10 projects run with `python -m benchmarks semantic --json /tmp/sem.json` . 5. Slow tier: `python -m pytest -m slow -q -p no:cacheprovider` (~19 min) → `870 passed` .

Done when: you ran all five once and recorded the five results under this task.

W-02 — Find why the checker gate keeps only 4 annotations on the resolver pair

- **Status:** open
- **Priority:** P0
- **Area:** Backend
- **Readiness:** ready

Context: Why does the type-checker gate keep only 4 annotations on the full resolver pair? Investigation, not a feature. Background: `python -m benchmarks.pair tests/pairs/resolver_extended_untyped.py.txt tests/pairs/resolver_extended_typed.py.txt --engine --min-con` printed `parameters_and_returns.correct = 4` , `variables = 0` ; the same run without `--checker` gives 329 and 185. On a 241-line slice the gate accepts everything (18 of 18), and `Checker.regresses` answers in ~1.5 s, so the gate is not simply broken. Steps: 1. Copy `/tmp/gated.py` from below to `scripts/gated_conversion.py` (it runs `fix_source(... engine=True, checker=build_checker(root=Path.cwd()))` on the full resolver untyped file and prints seconds, `len(applied)` , `len(skipped)` and `Counter(s.code.value for s in skipped)`). Run it in the background. 2. If almost everything is skipped with code `checker-rejected` : take ONE skipped fix, apply it alone to the original source (`Checker.new_errors(filename= "m.py", before=original, after=with_one_fix)`) and print what pyright says. Two outcomes: the single fix really adds an error (then the gate is right and the annotation is wrong — list the patterns) or it adds none (then the batch logic in `runner._degrade` / `_attempt` is wrong — read how `before` is chosen after each accepted half; `before` must be the source *after* the earlier accepted halves, not the original). 3. If the run ends in few skips but few applies, read `res.rounds` and `FixSettings.max_rounds` (default 3): the fixpoint may stop early.

Done when: you wrote the cause in 5 lines under this task and, if it is a bug in `runner.py` , fixed it with a test in `tests/test_rules_runner.py` that uses a stub `Checker` rejecting exactly one fix. Acceptance after a fix: with `--engine --checker` , resolver pair exact ≥ 250 of 557 and contradictions ≤ half of the 94 without the checker.

P-04 — "Tainted" parameters (83): types that only usage reveals (`x + 1` , `x.upper()` , `for _ in…

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: "Tainted" parameters (83): types that only usage reveals (`x + 1` , `x.upper()` , `for _ in x`) — capability contracts (F-13) and backward reconciliation (F-22).

Steps: *to be written*

Done when: *to be written*

P-05 — "No candidate" (72): the inference engine proposes nothing; measure which shapes and…

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: "No candidate" (72): the inference engine proposes nothing; measure which shapes and extend the engine or the negotiation seeds (calls into stubs F-15).

Steps: *to be written*

Done when: *to be written*

P-06 — "Call sites unknown / uncorroborated / escapes" (130): fixtures are closed worlds; add…

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: "Call sites unknown / uncorroborated / escapes" (130): fixtures are closed worlds; add `--closed-world` assumptions for the benchmark and measure how much of this is the open-world refusal.

Steps: *to be written*

Done when: *to be written*

P-07 — Name-agnostic scoring of synthesized Protocols and TypeVars (the author's…

- **Status:** open
- **Priority:** P1
- **Area:** Testing / QA
- **Readiness:** needs recipe

Context: Name-agnostic scoring of synthesized Protocols and TypeVars (the author's `LocalMultiplierProtocol` vs ours): compare members and signatures, not names.

Steps: *to be written*

Done when: *to be written*

P-09 — Round-trip gate: `fix(untype(typed))` against `typed` per pair, baseline stored, gate…

- **Status:** open
- **Priority:** P1
- **Area:** Testing / QA
- **Readiness:** needs recipe

Context: Round-trip gate: `fix(untype(typed))` against `typed` per pair, baseline stored, gate fails on lower recall or any contradiction.

Steps: *to be written*

Done when: *to be written*

P-10 — Import parity: every name the written annotations use is imported (M3-13), with the typed…

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: Import parity: every name the written annotations use is imported (M3-13), with the typed twin's import style (M3-08); measured as the missing-import count of the pair.

Steps: *to be written*

Done when: *to be written*

T-01 — Dunder parameters from the language (`autotyping --annotate-magics`):…

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: Dunder parameters from the language (`autotyping --annotate-magics`): `__exit__` / `__aexit__` take `type[BaseException] | None` , `BaseException | None` , `TracebackType | None` ; we annotate only their *result*. Declaration contracts (`Origin.protocol`) with the `types.TracebackType` import.

Steps: *to be written*

Done when: *to be written*

T-02 — Copy annotations from the base-class method of the same name (`infer-types` ; this is…

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: Copy annotations from the base-class method of the same name (`infer-types` ; this is M2-07 override awareness): a declaration contract from the overridden definition when the base is in the module, the project (`ProjectIndex`) or a typeshed/ `py.typed` stub; never contradict the base.

Steps: *to be written*

Done when: *to be written*

T-03 — Apply a sibling stub (`retype` , which is unmaintained since 2022): when `foo.pyi` sits…

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

- **Readiness:** needs recipe

Context: Apply a sibling stub (`retype` , which is unmaintained since 2022): when `foo.pyi` sits next to `foo.py` , or typeshed `stubs/<dist>` covers the module, seed its parameter/return/variable annotations as `declared` and write them with our lossless writer (`astart fix --apply-stubs`). Also the best oracle corpus for P-series work: untyped sdist source + the typeshed stub as the typed twin.

Steps: *to be written*

Done when: *to be written*

T-08 — Library return types from typeshed (`typeshed_client` , which `scripts/generate_libkw.py` ...

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** ready

Context: Library return types from typeshed (`typeshed_client` , which `scripts/generate_libkw.py` already uses offline): generate `rules/librets.py` the way `libkw.py` is generated — for every stdlib function and method with **one** signature (no `@overload`), no type variables and a plain return type (`str` , `int` , `bool` , `bytes` , `None` , `list[str]` , ...) record `callee → (parameter types, return type)` . Feed it to the derived nodes (F-15): `os.path.join(a, b)` is `str` once `a` and `b` are `str` ; overloaded and generic callees stay opaque. Use the resolver's version/platform evaluation (`get_search_context(version=…)`) so a branch that does not apply is not read. Acceptance: the dev pair's "unprovable Call" count drops, 0 new contradictions on both corpora, generation date and typeshed version recorded in the file like `libkw` .

W-03 — Stop `object` leaking into `--engine` annotations

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** ready

Context: Repro: `PYTHONPATH=src:. python scripts/pair_wrong.py dev/untyped.py dev/typed.py --engine --min-confidence 0.5` prints `... wrote dict[object, object] | None` . `engine_hints` filters with `hints.is_allowed_source` , which must reject `object` , so a different path wrote it. Steps: 1. Run the repro on a tiny module containing the `ExtendedDataclassStress` class from `dev/untyped.py` until the output still shows `object` . 2. Find who wrote it: `Hint.basis` values are in `HintBasis` (`hints.py`); print `facts.hints.index().hint_for(node=...)` for the parameter. 3. Add the missing check at that place (probably the dataclass-field path or `variables.engine`), with a test that fails first.

Finding (2026-11-08, partial): `engine_hints` (src/astart/rules/engine_hints.py) already rejects any annotation with `object` / `Any` anywhere (`is_allowed_source` walks every nested `Name`). A 9-line repro (class with `lookup=None` stored as `self.lookup = lookup or {}`) does NOT leak, so the leak needs the full `dev/untyped.py` . Next step: run `fix_source(...)` on the full file, loop over `res.applied` and print the fix whose text contains `object` (the objects in `res.applied` are listed by `dir(res.applied[0])` ; print one first). The writer is then one of: `variable_hints` / `engine_variables` (variables do not go through `engine_hints`), the dataclass-field path, or a hint index basis that builds annotations from `container_risks` . Put the `banned_names` check in that writer.

Done when: `pair_wrong.py ... --engine` prints no line containing `object` or `Any` , and the pair numbers did not drop by more than 3.

W-04 — Remove the last `typeshed_client` use from `generate_libkw.py`

- **Status:** open
- **Priority:** P1
- **Area:** DevOps / Release
- **Readiness:** ready

Context: Finish removing `typeshed_client` from `scripts/generate_libkw.py` (it is the last mention; the runtime never imports it). The script walks a typeshed root. `astart.stubs.find_typeshed()` finds one (it found basedpyright's copy), but `collect(root=…)` produced keys with a `stdlib.` prefix because it expected `root` to be the *contents* of the bundle used before. Steps: 1. In `main()` use `find_typeshed(explicit=Path(argv[1]) if len(argv) > 1 else None)` ; print an error and return 1 when it is `None` . 2. In `collect` / `_walk_body` make `root / "stdlib"` the base directory when it exists, so a module key is `os.path.join` , not `stdlib.os.path.join` ; third-party `stubs/<dist>/<pkg>` entries keep working only for packages in `THIRD_PARTY` . 3. Replace the `typeshed_client` version line in the footer by `# from the typeshed copy at {root}.` . 4. Run `PYTHONPATH=src python scripts/generate_libkw.py > /tmp/libkw_new.py` (takes ~5 min; background). Compare only the `SIGNATURES` dictionary:
`diff <(sed -n '/^SIGNATURES/,$p' /tmp/libkw_new.py | grep -v '^# ') <(sed -n '/^SIGNATURES/,$p' src/astart/rules/libkw.py | grep -v`

Done when: the diff is empty or only adds/removes entries (no changed keys); then copy the new file over `src/astart/rules/libkw.py` , run `tests/test_rules_libkw.py tests/test_rules_library_calls.py` and FAST. If the diff shows renamed keys you cannot explain, keep the old file and

stop.

W-06 — Burn down unprovable calls in the negotiator

- **Status:** open
- **Priority:** P1

- **Area:** Backend
- **Readiness:** ready

Context: Known sub-cases, easiest first: 1. Calls on a receiver whose type is known (`value.strip()` , `items.copy()` , `d.get(k)`): the derived-node templater already allows attribute calls (`_Templater.convert`); check with `scripts/opacity_report.py` which still fail and extend `provable._Typer` method tables (`str_to_str` , ... in `provable.Constants`) with the missing methods. One table row + one test each. 2. Library functions returning a class from `known_imports` (`Path` , `Decimal` , `UUID` , `date` ...): in `stubs/returns.py` `_plain` allow Name ids that are keys of `imports.Constants.known_imports` ; `ensure_names` inserts the import. Test: `shutil.which` is excluded (returns `str | None` , overloaded?) — check with `host_resolver().resolve("pathlib.Path")` first. 3. `list(x)` , `sorted(x)` , `dict(x)` , `sum(x)` of node arguments: already typed by `provable._Typer.special_calls` when the argument type is known; if not derived, find out why in `_Templater` (usually a keyword argument — keywords make `_field` reject the call; allow `keyword` nodes whose value converts). Skip: `super().__new__` , `functools.partial` , `reduce` , generics with `T` .

Done when: unprovable-call count drops by ≥ 10 and the pair numbers rise.

W-07 — Burn down unbindable calls and escaping functions

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** ready

Context: For the first: run `CallBindings( module=..., counts=...).get(call=c)` on the 16 calls (`opacity_report.py` prints samples) and list why each is refused (decorated callee, `*args` , method on an unknown receiver, dynamic marker). Fix only the refusals that are provably safe, e.g. a decorator that is `functools.wraps` -based, using the danger matrix `tests/test_rules_kwargs_danger.py` (it must stay green). For the second, only treat a function as escaping when the name is read as a value *outside* a decorator position: `@decorator` applied to a function makes the *decorated* function reachable, not the decorator's parameters unknown.

Done when: both counts drop and `tests/test_rules_kwargs_danger.py` passes.

W-08 — Let the negotiator write default-literal parameters (F-37)

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** ready

Context: F-37: default-literal parameters in the negotiator (70 annotations the hint index writes and the negotiator does not; 60 exact, 3 wrong). Steps: 1. In `negotiation/policy.py` `_supported` , a candidate whose origins are exactly `{Origin.default}` and the engine agrees (`corroborated`) is already accepted. The gap is *opaque callers*: `builders._calls` marks a parameter opaque when any call to its function cannot be bound. Add a second set `Built.caller_opaque` (parameters blocked ONLY for that reason) and let `blocked_nodes` ignore it when the belief's candidates are all `Origin.default` . 2. Keep the tiered confidence: `confidence.by_literal` (bool/float 0.99, int 0.72, str 0.69) decides in `facts._has_inferred_hint` / `offer_confidence` ; set the hint's `HintBasis.default_literal` , not `negotiated` , so the tier applies. 3. Test: `def f(x=1): return x` with an unbindable call `f(*args)` elsewhere and engine `{"f.x": "int"}` → `x: int` at `--min-confidence 0` , nothing at 0.8.

Done when: shadow `only_existing` for basis `default_literal` falls from 70 to < 20 and `only_negotiated.wrong` stays 0.

W-11 — Annotate a method that returns its own class (`Self` or quoted name)

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: Today `-> URIBuilder` inside `class URIBuilder` is refused (it would raise `NameError`). Add the alternative: in `supplement.negotiated_hints` and `engine_hints` , when the only blocker is `bound_when_evaluated` and the name is the *enclosing class* of the method, write `Self` if `facts.target` allows it (3.11+; `PythonTarget.self_type`), else the quoted string `'URIBuilder'` . The writer must import `Self` (`known_imports` has it).

Test: builder pattern returns `URIBuilder(...)` → `-> Self` on 3.11+, `-> 'URIBuilder'` with `--target-version 3.10` ; import the fixed module in the test (see `tests/test_rules_negotiated_fix.py::_runs`).

Steps: *to be written*

Done when: *to be written*

W-12 — Annotate instance attributes from negotiation

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: Instance attributes: `Built.attributes` already maps each attribute node to its first write (`self.x = ...`). Add to `negotiation/variables.py` `variable_hints` a loop over `built.attributes` that accepts with `policy.accept(kind="attribute", ...)` and returns hints keyed by that assignment's `node_key` ; then make `appliers.add_attribute_annotation` also accept `basis is HintBasis.negotiated` (today it only accepts `attribute_assignments`) and read the hint from `facts.variables` . Fact: extend `facts._has_inferred_hint` 's Attribute branch the same way. Test: `self.x = x` with `x: int` from callers → `self.x: int = x` . Target: the 47 `self.x: T = value` of the dev pair. Class-level `x: T` (36 in the dev pair) stays open — separate task after this one.

Steps: *to be written*

Done when: *to be written*

T-04 — Name heuristics as *calibrated candidates* (`infer-types` `is_` / `has_` → `bool` ,...

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** needs recipe

Context: Name heuristics as *calibrated candidates* (`infer-types` `is_` / `has_` → `bool` , `autotyping --guess-common-names` `verbose` → `bool` , user maps `--annotate-named-param`): measure each name pattern on both corpora with the X-05 script; enable only the passing ones; user-configured maps (`[tool.astart.names]`) enter as `declared` because the user asserted them.

Steps: *to be written*

Done when: *to be written*

T-05 — Ingest external suggestions (`autotyping --pyanalyze-report`): `--suggestions FILE` reads...

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** needs recipe

Context: Ingest external suggestions (`autotyping --pyanalyze-report`): `--suggestions FILE` reads pyanalyze/MonkeyType/pytype JSON and feeds it to the negotiator as another *corroborator* next to the engine (never as a source of types); `--only-without-imports` equivalent.

Steps: *to be written*

Done when: *to be written*

T-06 — Generators: `infer-types` returns `Iterator` for any `yield` , which `autotyping` itself...

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** needs recipe

Context: Generators: `infer-types` returns `Iterator` for any `yield` , which `autotyping` itself calls imprecise. Keep refusing until M4-06 can state `Iterator[T]` / `Generator[Y, S, R]` with the element types from the yield contracts.

Steps: *to be written*

Done when: *to be written*

T-09 — Optional runtime use of `typeshed_client` in `--project` mode: replace the hand-written...

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** needs recipe

Context: Optional runtime use of `typeshed_client` in `--project` mode: replace the hand-written stub reader in `project/libraries.py` where its `Resolver` is available (third-party `*-stubs` and `types-*` packages from the environment), keeping the current reader as the fallback so the tool still works without it.

Steps: *to be written*

Done when: *to be written*

W-05 — Sync CHANGELOG, evidence ladder, architecture and README with the code

- **Status:** open

- **Status:** open
- **Priority:** P2
- **Area:** Documentation
- **Readiness:** ready

Context: No code. 1. `CHANGELOG.md` : add `## 2026-11-08` with one bullet each for: contract negotiation (default on, `--no-negotiate`), `--annotate-variables` , `--engine` conversion mode, the `astart.stubs` typeshed reader, library return types, the pair benchmark and `tests/pairs` , `scripts/opacity_report.py` . 2. `docs/evidence-ladder.md` : add the origins `protocol` , `calibrated` , `engine` and the rule "engine = lowest tier, checker gate vetoes" (copy wording from `docs/negotiation.md`). 3. `docs/architecture.md` : it is generated from module docstrings (Q-23); add one paragraph for each new package (`negotiation` , `stubs`) and for `rules/engine_hints.py` , `rules/library_calls.py` . 4. `README.md` "Limits": replace the stale cross-module/typeshed bullets with what `--project` and `astart.stubs` do now.

Done when: `python -m pytest tests/test_readme_flags.py tests/test_rule_docs.py -q -p no:cacheprovider` passes.

W-09 — Annotate the parameters of `__exit__` and `__aexit__`

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** needs recipe

Context: Table (put in `negotiation/builders.py` `Constants.dunder_parameters`): `{"__exit__": ("type[BaseException] | None", "BaseException | None", "TracebackType | None"), "__aexit__": same}` . In `_declare` , for a function in this table that is a method, seed each of its three non-receiver parameters that are unannotated with `Origin.protocol` (a proof) via `_seed_type` . The type text `TracebackType` needs `from types import TracebackType` : first do H-07 (add `TracebackType: "types"` to `imports.Constants.known_imports`).

Test: a class with `def __exit__(self, kind, value, trace): ...` gets all three annotations and one import; an already-annotated method is untouched.

Steps: *to be written*

Done when: *to be written*

W-10 — Copy annotations from the base class's method

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** needs recipe

Context: In `builders.py` add `_inherit(unique)` called after `_declare` loop: for every method whose class has exactly one base that is a class of this module (use `walk_classes`) defining a method with the same name and the same parameter names, seed this method's unannotated parameters and return with the base's annotation text and `Origin.protocol` . Refuse when the base method is itself unannotated, when parameter names differ, or when the annotation names a class defined later (`bound_when_evaluated` already guards the final write). Test: `class A: def f(self, x: int) -> str` and `class B(A): def f(self, x)` → `B.f.x: int` , `B.f->: str` .

Steps: *to be written*

Done when: *to be written*

W-13 — Type generators as `Iterator[...]`

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** needs recipe

Context: Today `builders._body` marks the return opaque when the function has `yield` . Implement only the sound core: for a function whose every `yield` is a bare `yield <expr>` (no `yield from` , no send value used) and which has no `return <value>` , seed nothing but add a hidden node `f.<yield>` fed by each yielded expression (like `_flow`); then report `f->` as `Iterator[<yield type>]` (`collections.abc.Iterator` , already in `known_imports`) via a new relation `iter_of` in `transfer.Constants.wrappers` . Async generators → `AsyncIterator` . Refuse everything else. Tests: two int yields → `Iterator[int]` ; mixed `int` / `str` → `Iterator[int | str]` ; `yield from` → no annotation.

Steps: *to be written*

Done when: *to be written*

W-14 — Annotate class-level and module-level variables

- **Status:** open
- **Priority:** P2
- **Area:** Backend

- **Readiness:** needs recipe

Context: Class-level and module-level variables (`x: T = value` outside functions; 36 class-level in the dev pair, e.g. `threshold: Final[int] = 5` is another rule — skip `Final`). Today `builders` only creates nodes inside functions. Add a module/class scope in `ContractBuilder` : assignments with a single `Name` target in the module body or a class body (not in `__slots__` , not in a dataclass? — dataclass fields need annotations already) become nodes with path `Module.<name>` / `Class.<name>` , flows as in `_assignment` . Only emit when `annotate_variables` or the style signal is on. The write-back is the same `add_variable_annotation` . Test: module `LIMIT = 5` → `LIMIT: int = 5` .

Steps: *to be written*

Done when: *to be written*

T-07 — `typing_utils.issubtype` as a differential oracle for `TypeAlgebra.intersect` and…

- **Status:** open
- **Priority:** P3
- **Area:** Testing / QA
- **Readiness:** needs recipe

Context: `typing_utils.issubtype` as a differential oracle for `TypeAlgebra.intersect` and `recovery.relation` on builtin/ `typing` generics (dev-only; it works on runtime objects, so it cannot judge project classes, protocols, or text we cannot import, and its last CI covers Python 3.6–3.9).

Steps: *to be written*

Done when: *to be written*

Completed

ID	Task	Priority	Area
P-01	Pair benchmark and accepted baseline: `benchmarks/pair.py` , `tests/pairs/` ,…	P0	Testing / QA

12. Typer and small tasks

Scope: Small self-contained tasks that need no design.
ID prefixes: Q, X

▶ Background — 18. Atomic backlog for junior agents

▶ Background — 18.0 Working rules (read first)

▶ Background — 18.1 Typer extensions (all in `src/astart/rules/provable.py` , class `_Typer`)

▶ Background — 18.2 Gold and regression tests (no production code)

▶ Background — 18.5 Detection rules (copy the TYP005 recipe)

▶ Background — 18.6 Ant-farm refactor, stages 1–2 (see the design in the 2026-11-07

▶ Background — 18.8 Copy-paste tasks (no design needed)

▶ Background — 18.9 Typing-inference tasks (verified against the code on 2026-11-07)

▶ Background — 18.11 Gaps and findings from the external corpus (2026-11-08)

▶ Background — 18.10 Docstring and whitespace healing (follow-ups)

Q-28 — `--format sarif` for `astart check` (M7-09)

- **Status:** open
- **Priority:** Untriaged
- **Area:** Uncategorized
- **Readiness:** needs recipe

Context: Output a JSON document `{"version":"2.1.0","runs":[{"tool":{"driver":{"name":"astart", "rules":[...]}},"results":[...]}]}` ; each finding becomes a result with `ruleId` (the code), `level` (`error|warning|note` from severity `error|warning|info`), `message.text` , and one `locations[0].physicalLocation` with `artifactLocation.uri` and `region.startLine` . Copy `render_json(*, results, catalog)` in `src/astart/cli/render.py` as `render_sarif` with the same signature, and add `sarif` to the `--format` choices in `__main__.py` and to the dispatch that calls `render_json` . Test: parse the output and assert those keys for one finding.

Steps: *to be written*

Done when: *to be written*

Q-33 — TYP002 `typing.no-object-annotation` (M8-03): flag any annotation that is exactly…

- **Status:** open
- **Priority:** Untriaged
- **Area:** Uncategorized
- **Readiness:** needs recipe

Context: TYP002 `typing.no-object-annotation` (M8-03): flag any annotation that is exactly `object` or `Any` (bare names). Subject kind `Parameter` and `Function` already exist: add a fact `annotation.is_object_or_any` that unparses `arg.annotation` / `function.returns` and compares with `{"object","Any"}`. No fix.

Steps: *to be written*

Done when: *to be written*

Q-34 — TYP006 `typing.bare-generic` (new code): flag annotations that are exactly `list`,…

- **Status:** open
- **Priority:** Untriaged
- **Area:** Uncategorized
- **Readiness:** needs recipe

Context: TYP006 `typing.bare-generic` (new code): flag annotations that are exactly `list`, `dict`, `set`, `tuple`, `frozenset` without subscripts. Same recipe as Q-33 with a different set. No fix.

Steps: *to be written*

Done when: *to be written*

Q-35 — PRS001 (M8-05): when the file does not parse, `check` should report ONE finding `PRS001`…

- **Status:** open
- **Priority:** Untriaged
- **Area:** Uncategorized
- **Readiness:** needs recipe

Context: PRS001 (M8-05): when the file does not parse, `check` should report ONE finding `PRS001` at line 1 instead of crashing or reporting nothing. Find where `check` handles `SyntaxError` (`grep -n "SyntaxError" src/astart/rules/engine.py src/astart/cli/*.py`); add the finding there. Test with `"def (:\n"`.

Steps: *to be written*

Done when: *to be written*

Q-38 — Add `reliability: float` to `_Site` in `hints.py` defaulting to `1.0` (`0.0` where…

- **Status:** open
- **Priority:** Untriaged
- **Area:** Uncategorized
- **Readiness:** needs recipe

Context: Add `reliability: float` to `_Site` in `hints.py` defaulting to `1.0` (`0.0` where `definite=False`). Behaviour must not change: run the full fast tier and the gold file before and after, outputs identical. (Do this only after the lead says `hints.py` is free.)

Steps: *to be written*

Done when: *to be written*

Q-57 — Return-type characterization through the fixer

- **Status:** open
- **Priority:** Untriaged
- **Area:** Uncategorized
- **Readiness:** needs recipe

Context: Create `tests/test_rules_return_matrix.py` using the `_fixed` helper from 18.0 and these (source → expected substring) pairs: `"def f():\n return 1\n"` → `"def f() -> int:"` ; `"def f():\n return 1.5\n"` → `"-> float:"` ; `"def f():\n return 'a'\n"` → `"-> str:"` ; `"def f():\n return [1, 2]\n"` → `"-> list[int]:"` ; `"def f():\n return {'a': 1}\n"` → `"-> dict[str, int]:"` ; `"def f(x):\n if x:\n return 1\n"` → `"-> int | None:"` ; `"def f():\n return None\n"` → `"-> None:"` ; `"def f():\n pass\n"` → `"-> None:"` . Refusals (assert the substring `"->"` is NOT in the `def` line): `"def f(x):\n return x\n"` becomes generic via TypeVar, so use instead

`"def f(x):\n return x.y\n"` . Expected: all pass today; if one does not, paste the real output under the task and mark it `BLOCKED` (do not change production code). **BLOCKED 2026-11-08** — 7 of 8 rows pass; the `pass` -only body produces `def f():\n pass\n` with no `-> None:` (real output verified). The row is pinned with `xfail(strict=True)` in `tests/test_rules_return_matrix.py` ; the `return x.y` refusal passes.

Steps: *to be written*

Done when: *to be written*

Q-58 — Parameter-hint characterization

- **Status:** open
- **Priority:** Untriaged
- **Area:** Uncategorized
- **Readiness:** needs recipe

Context: Create `tests/test_rules_param_matrix.py` with the `_fixed` helper and these pairs: `"def f(x=1):\n return x\n"` → `"x: int = 1"` ; `"def f(x='a'):\n return x\n"` → `"x: str = 'a'"` ; `"def f(x=1.5):\n return x\n"` → `"x: float = 1.5"` ; `"def f(x=True):\n return x\n"` → `"x: bool = True"` ; `"def f(x):\n return x\n\nf(1)\nf(2)\n"` → `"x: int"` ; `"def f(x):\n return x\n\nf(1)\nf('a')\n"` → NOT contains `"x: int"` and NOT contains `"x: str"` . Same BLOCKED rule as Q-57. **BLOCKED 2026-11-08** — 5 of 6 rows pass; the `f(1)` / `f(2)` row now synthesizes `x: T` via TypeVar (M5-14…M5-18 postdates this spec) instead of the expected `x: int` . Real output: `T = TypeVar('T')` + `def f(x: T) -> T:` . Row pinned with `xfail(strict=True)` in `tests/test_rules_param_matrix.py` ; the mixed-call refusal passes.

Steps: *to be written*

Done when: *to be written*

Q-61 — Returns / Raises stubs

- **Status:** open
- **Priority:** Untriaged
- **Area:** Uncategorized
- **Readiness:** needs recipe

Context: Reuse `docs_missing_returns.py` and `docs_missing_raises.py` (DOC004 / DOC006). Add rules + fixes that append a `Returns:\n Describe the result.` (only when a `return <value>` exists) or `Raises:\n <Exc>: Describe when <Exc> is raised.` section using `docstring_edit._new_section` 's placement logic: generalise it to take the title (`Args` / `Returns` / `Raises`) and the entry lines. Tests mirror Q-60.

Steps: *to be written*

Done when: *to be written*

Q-64 — Docstring reflow with `textwrap`

- **Status:** open
- **Priority:** Untriaged
- **Area:** Uncategorized
- **Readiness:** needs recipe

Context: Add `docs.wrap-docstring` (rule `DOC008` , info): wrap prose paragraphs (not section entries, not code blocks starting with `>>>` or indented 4+ extra spaces) to 79 columns with `textwrap.fill(width=79 - len(indent), initial_indent=indent, subsequent_indent=indent)` . Fact `docstring.has_long_prose` . Off by default (needs a `--wrap-docstrings` flag, same plumbing as `--source-root`).

Steps: *to be written*

Done when: *to be written*

Q-68 — Kill the ~50k-line cliff (M7-01 evidence from Q-31)

- **Status:** open
- **Priority:** Untriaged
- **Area:** Uncategorized
- **Readiness:** needs recipe

Context: On the 50,763-line `profile_perf` workload `check` alone exceeded 30 min (> 85× the 12k `check` for 4.2× the lines) and `fix` multiplies it via `max_rounds=3` + `_degrade` halving. `cProfile` shows the dominant cost is `hints.index → _candidates → CommentedModule.convert → semantic/analyzer.convert → _graph/conversion → backend/checkpoint._infer` — the whole-module graph conversion runs once per hint-index build and is itself superlinear. Recipe: (1) confirm whether `convert` results are cached per module content across `check` / `fix` rounds and `_degrade` splits (the shared `hint_cache` should make repeats cheap — verify it actually does); (2) if not, key/cache the conversion so `_attempt` / `_rematch` / `module_facts` on the same source reuse it; (3) if the conversion itself is the superlinear part, bound or skip it for modules over a size threshold and record the recall trade-off; (4) re-run `benchmarks/profile_perf.py --lines 50000 --json` and replace the `> 30 min` lower bound in `docs/benchmarks.md` . Constraints: no precision loss on the 128-module recovery gate (rerun `recovery --json` + `gate` before/after) and no

lower bound in `docs/benchmarks.md`. Constraints: no precision loss on the 126-module recovery gate (rerun `recovery --json` + `gate` before/after) and no behaviour change below the threshold.

Steps: *to be written*

Done when: *to be written*

X-02 — P1

- **Status:** open
- **Priority:** Untriaged
- **Area:** Uncategorized
- **Readiness:** needs recipe

Context: Contradictions that remain (10 of 2 788). Run `benchmarks recovery --corpus tests/corpus_ext`, list `wrong` per module, classify each (inference bug vs scoring artefact), fix or document each in `docs/benchmarks.md`. Then store the external run as `benchmarks/baseline_ext.json` with a gate (precision ≥ 99.5 %, 0 crashes).

Steps: *to be written*

Done when: *to be written*

X-03 — P1

- **Status:** open
- **Priority:** Untriaged
- **Area:** Uncategorized
- **Readiness:** needs recipe

Context: `Self` and TypeVar returns for classmethods, `__enter__` and `__aiter__` (`-> Self`, `-> T`); the fixer writes the class name or `_Self` / `__Self` spellings that clash with the original (`Path.cwd`, `_SignalReceiver.__aiter__`). Tie to M5-18.

Steps: *to be written*

Done when: *to be written*

X-04 — P1

- **Status:** open
- **Priority:** Untriaged
- **Area:** Uncategorized
- **Readiness:** needs recipe

Context: Names imported from other modules cannot be judged by `relation()` ("unresolved": 212 compatible/unresolved of 2 774 written). Resolve them through the project import graph (`ProjectIndex`, M3) or typeshed so the scorer and the fixer share one notion of subtype.

Steps: *to be written*

Done when: *to be written*

X-05 — P1

- **Status:** open
- **Priority:** Untriaged
- **Area:** Uncategorized
- **Readiness:** needs recipe

Context: Calibrate more candidate categories with the same method as `Constants.calibrated_returns` (script idea: for every unaccepted engine candidate, compare with the original by reason × kind × shape; accept a category only at ≥ 97 % exact **and** 0 contradictions, n ≥ 100, on stdlib *and* external corpus). Measured today (n = 4 985): only syntactic `bool` returns qualified. Candidates that failed and must stay refused: `x=None` params (`int | None` guessed, 9.4 % exact, n = 1 056), generators (0.5 %), tuples (6.7 %), `Sequence` / `Callable` / `Iterable` params (0–1 %), `object` params (0 %). Re-run after M4-06/M4-07 land.

Steps: *to be written*

Done when: *to be written*

X-06 — P1

- **Status:** open
- **Priority:** Untriaged
- **Area:** Uncategorized
- **Readiness:** needs recipe

Context: Per-module `--jobs` in `astart fix` itself (the benchmark harness has it; the CLI does not): one process per file, same deterministic output order (M7-04/M7-05).

Steps: *to be written*

Done when: *to be written*

X-07 — P2

- **Status:** open
- **Priority:** Untriaged
- **Area:** Uncategorized
- **Readiness:** needs recipe

Context: Grow the external corpus: more distributions (async, data, CLI, ORM), a held-out split that no rule may be tuned on, and a second Python-version build. Keep `tests/corpus_ext/MANIFEST.json` authoritative.

Steps: *to be written*

Done when: *to be written*

X-08 — P2

- **Status:** open
- **Priority:** Untriaged
- **Area:** Uncategorized
- **Readiness:** needs recipe

Context: Missed-annotation reasons on the external corpus (12 789 originals): `no candidate` 3 686, `tainted` 2 371, `None default` 1 083, `uncorroborated` 558, `call sites unknown` 476, `return type not provable` 437, `contains object` 356, `declaration body` 281, `generator` 227. Each is a roadmap item: M4-02/M2-09, M4-12 follow-up for `x=None` (needs body-usage evidence, not call sites, for public library APIs), M4-06, M3-03.

Steps: *to be written*

Done when: *to be written*

Completed

ID	Task	Priority	Area
Q-01	String methods that return `str`	Untriaged	Uncategorized
Q-02	String methods that return `bool` / `int` : `startswith endswith isdigit isalpha isalnum…	Untriaged	Uncategorized
Q-03	`str % anything` → `str` (printf formatting)	Untriaged	Uncategorized
Q-04	List concatenation: in `_binop` , when both sides type to the same `list[...]` string and…	Untriaged	Uncategorized
Q-05	Subscripts	Untriaged	Uncategorized
Q-06	`abs` , `round` , `min` , `max` , `sorted`	Untriaged	Uncategorized
Q-07	`list(x)` , `set(x)` , `tuple(x)` with a typed argument: `list(list[T])` → `list[T]` ;…	Untriaged	Uncategorized
Q-08	`dict.get` / `list.copy` / `dict.keys` are NOT in scope; add a comment-free refusal test…	Untriaged	Uncategorized
Q-09	Add family `variadic` (also add it to the `expected` set in…	Untriaged	Uncategorized
Q-10	Add family `attribute_read` with 6 cases mirroring `tests/test_rules_attribute_reads.py` …	Untriaged	Uncategorized
Q-11	Add family `comprehension` with 6 cases mirroring…	Untriaged	Uncategorized
Q-12	Add family `multi_binding` with 6 cases mirroring…	Untriaged	Uncategorized
Q-13	CRLF: new `tests/test_rules_line_endings.py`	Untriaged	Uncategorized
Q-14	Hostile inputs: new `tests/test_rules_hostile_inputs.py` , parametrized over: empty…	Untriaged	Uncategorized
Q-15	Determinism across hash seeds: new `tests/test_rules_determinism.py`	Untriaged	Uncategorized
Q-16	Convention guard (CC-06): new `tests/test_conventions.py` with three tests that scan…	Untriaged	Uncategorized
Q-17	Constants guard: in the same file, assert that every class named `Constants` in…	Untriaged	Uncategorized
Q-18	README flag guard: new `tests/test_readme_flags.py`	Untriaged	Uncategorized
Q-19	`astart --version`	Untriaged	Uncategorized

Q-19	`astart --version`	Untriaged	Uncategorized
Q-20	`astart rules` : new subcommand printing one line per rule: `CODE key severity title` ,…	Untriaged	Uncategorized
Q-21	`astart explain CODE` : prints the rule's `rationale` , then each example's source with…	Untriaged	Uncategorized
Q-22	`docs/rules.md` generated by a new script `scripts/generate_rule_docs.py` (iterate the…	Untriaged	Uncategorized
Q-23	`docs/architecture.md` : one paragraph per module in `src/astart/rules/` (`ls` it; read…	Untriaged	Uncategorized
Q-24	`CHANGELOG.md` : one `## 2026-11-07` heading, bullets copied from the entries in section…	Untriaged	Uncategorized
Q-25	`.pre-commit-hooks.yaml` (M7-11): one hook, id `astart-fix` , `entry: astart fix --check` ,…	Untriaged	Uncategorized
Q-26	`.github/workflows/ci.yml` : on push/pull_request, Python 3.12, steps: install with `pip…	Untriaged	Uncategorized
Q-27	`--exclude GLOB` for `astart check` and `astart fix` (repeatable)	Untriaged	Uncategorized
Q-29	Grow the corpus (M1-07 beyond 113): copy 15 stdlib modules not yet present to…	Untriaged	Uncategorized
Q-30	Five more semantic projects (M1-12): add entries to `benchmarks/projects.json` copying…	Untriaged	Uncategorized
Q-31	The ≈ 50k-line throughput row (M1-10): concatenate corpus files into one 50k-line module…	Untriaged	Uncategorized
Q-32	`docs/benchmarks.md` : add a "How to refresh the baseline" section with the three exact…	Untriaged	Uncategorized
Q-36	Read `HintIndex._judge_pass` , `_call_sites` and `_judge_call_sites` in…	Untriaged	Uncategorized
Q-37	Create `src/astart/rules/slots.py` with a `SlotTable` class: `deposit(*, slot: str, kind:…	Untriaged	Uncategorized
Q-39	`tests/test_rules_typer_negatives.py` : 12 one-line refusal tests proving the typer…	Untriaged	Uncategorized
Q-40	`tests/test_rules_hint_cache.py` : assert two `fix_source` calls on identical text return…	Untriaged	Uncategorized
Q-41	Document every `Rejection` value: `grep -n "class Rejection"…	Untriaged	Uncategorized
Q-42	Document every `SkipCode` the same way (`docs/skip-codes.md`)	Untriaged	Uncategorized
Q-43	`README.md` : add a "Limits" section (five bullets, copied from the "Where it is far from…	Untriaged	Uncategorized
Q-44	Add `tests/test_cli_exit_codes.py` : `fix --check` on a clean file → 0; on a file with a…	Untriaged	Uncategorized
Q-45	Add `tests/test_cli_formats.py` : `check --format json` output parses with `json.loads` …	Untriaged	Uncategorized
Q-46	Add `tests/test_rules_variadic_methods.py` : the same cases as…	Untriaged	Uncategorized
Q-47	Add `tests/test_rules_default_expression_edges.py` : refusal tests for `x=-1` called with…	Untriaged	Uncategorized
Q-48	Create `tests/test_rules_typer_negatives.py`	Untriaged	Uncategorized
Q-49	Create `tests/test_rules_typer_positives.py` with the same shape as Q-48 but parametrize…	Untriaged	Uncategorized
Q-50	Create `.pre-commit-hooks.yaml`	Untriaged	Uncategorized
Q-51	Create `tests/test_conventions.py`	Untriaged	Uncategorized
Q-52	`README.md` : add a `## Limits` section just after the paragraph under the `astart fix` …	Untriaged	Uncategorized
Q-53	Create `CHANGELOG.md` with `# Changelog` , a `## 2026-11-07` heading, and these bullets:…	Untriaged	Uncategorized
Q-54	Create `tests/test_rules_typer_matrix.py` pinning every row of the table above that says…	Untriaged	Uncategorized
Q-55	`**` operator	Untriaged	Uncategorized
Q-56	Pin the gaps so closing them is visible: create `tests/test_rules_typer_gaps.py` with…	Untriaged	Uncategorized
Q-59	Write `docs/typing-inference.md` : copy the table at the top of this section verbatim, add…	Untriaged	Uncategorized
X-01	P0	Untriaged	Uncategorized
Q-60	DOC001 stub docstrings — DONE 2026-11-08 with a different design than first sketched	Untriaged	Uncategorized
Q-62	Run ruff on docstring content	Untriaged	Uncategorized
Q-63	Whitespace outside docstrings	Untriaged	Uncategorized
Q-65	Heal renamed parameters	Untriaged	Uncategorized
Q-66	Module docstring support: extend `_docstring()` in `rules/docs_fixes.py` to accept…	Untriaged	Uncategorized
Q-67	`--no-docs` flag (`astart fix --no-docs`) that excludes the `docs.*` fixes, for users who…	Untriaged	Uncategorized

13. Handoff batch

Scope: Tasks pre-selected for a second agent.
ID prefixes: H

▶ Background — 18.15 Handoff batch for a second agent (selected 2026-11-08)

H-01 — Reporters in the CLI (M7-09, Q-28, part of M7-16)

- **Status:** open
- **Priority:** P0
- **Area:** DevOps / Release
- **Readiness:** needs recipe

Context: `render_sarif`, `render_junit` and `render_github` already exist in `src/astart/cli/{sarif,junit,github}.py` with tests, but `--format` only offers `text` and `json` (`src/astart/__main__.py`, search `choices=("text", "json")`). Add the three choices to `check` and dispatch to the renderers in the same place `json` is dispatched (`cli/commands.py`, follow `render_json`). Read each renderer's signature first; do not change the renderers. Tests `tests/test_cli_reporters.py` through `main(argv=[...])` on a `tmp_path` file with one finding: sarif output parses as JSON with `version == "2.1.0"` and one result; junit parses with `xml.etree.ElementTree` and has one `testcase`; github lines start with `::`. README: document the three values (the README flag test fails until you do).

Steps: *to be written*

Done when: *to be written*

H-02 — `[tool.astart]` in the CLI (M7-07 user-visible half, M7-16)

- **Status:** open
- **Priority:** P0
- **Area:** DevOps / Release
- **Readiness:** needs recipe

Context: `cli/config.py` loads it and `cli/ignores.py` applies it, but nothing calls them. In `commands.py` discover `pyproject.toml` upward from the first path argument, load it, and let it supply `select`, `ignore`, `min-confidence`, `safety-ceiling`, `checker`, `exclude` and `target-version` when the matching flag was not given (a flag always wins). Tests `tests/test_cli_config_wiring.py`: config alone, flag overriding config, per-path override, no `pyproject.toml`, malformed config → exit 2 with a message on stderr.

Steps: *to be written*

Done when: *to be written*

H-08 — `astart untype` (P-08): the reverse of `fix`

- **Status:** open
- **Priority:** P0
- **Area:** Backend
- **Readiness:** needs recipe

Context: New `src/astart/rules/untype.py` `untype_source(*, source: str, variables: bool = True) -> str` removing parameter, return and (optionally) variable annotations through the lossless writer, keeping comments, layout and docstrings; an `x: int = 1` becomes `x = 1`, an `x: int` without a value is removed. CLI subcommand `untype` (files in place, `--diff`, `--check`, like `fix`). Tests `tests/test_rules_untype.py`: comments kept, defaults kept, `ast.dump(parse(untype(s)))` equals `ast.dump(parse(strip_annotations(s)))` with variables ignored, idempotent, round trip on `tests/pairs/*typed*` (record the remaining differences from the author's untyped twin under the task; do not try to match the hand-edited parts).

Steps: *to be written*

Done when: *to be written*

H-03 — Baselines in the CLI (M7-08)

- **Status:** open
- **Priority:** P1
- **Area:** DevOps / Release
- **Readiness:** needs recipe

Context: `cli/baseline.py` has `save_baseline`, `load_baseline`, `apply_baseline`. Add `--write-baseline FILE` and `--baseline FILE` to `check`: write the findings, or hide the findings already in the file and exit 0 when nothing new remains. Tests `tests/test_cli_baseline_wiring.py`: write, then clean re-run exits 0, then a new finding exits 1, a missing file exits 2.

Steps: *to be written*

Done when: *to be written*

H-04 — Catalog rules TYP002 and TYP006 (Q-33, Q-34, M8-03)

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: Follow recipe 18.5. TYP002 `typing.no-object-annotation` wraps the existing detector `rules/object_annotations.py`; TYP006 `typing.bare-generic` flags `list` / `dict` / `set` / `tuple` / `frozenset` annotations without subscripts. Detection only, no fix. Edit the JSON under `src/astart/rules/catalog/` by hand (`json.dumps(d, indent=2, sort_keys=True) + "\n"`; never regenerate it), update the expected code list in `tests/test_rules_end_to_end.py`, regenerate `docs/rules.md` with `PYTHONPATH=src python scripts/generate_rule_docs.py`. Every rule needs one triggering and one clean example (`verify_catalog`).

Steps: *to be written*

Done when: *to be written*

H-05 — PRS001 for files that do not parse (Q-35, M8-05): one finding at line 1 instead of an…

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: PRS001 for files that do not parse (Q-35, M8-05): one finding at line 1 instead of an `error:` line. Find the `SyntaxError` handling (`grep -n SyntaxError src/astart/rules/engine.py src/astart/cli/*.py`), reuse `rules/parse_module.py`. Keep the exit status 2. Test with `"def (:\n"` in `tests/test_rules_parse_rule.py`.

Steps: *to be written*

Done when: *to be written*

H-06 — Returns / Raises docstring stubs (Q-61, M8-07)

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: Generalise `docstring_edit._new_section` to take the title and entry lines, add DOC004 (`Returns:` with `Describe the result.`, only when a `return <value>` exists) and DOC006 (`Raises:` with `<Exc>: Describe when <Exc> is raised.`) as catalog rules with fixes in `docs_fixes.py`, reusing `docs_missing_returns.py` / `docs_missing_raises.py`. Tests mirror `tests/test_rules_docstring_stubs.py`; idempotence on three corpus files.

Steps: *to be written*

Done when: *to be written*

H-07 — Wider `known_imports` (M3-13)

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: Add to `rules/imports.py` `Constants.known_imports`: `Never`, `TypeGuard`, `TypeIs`, `ClassVar`, `Literal`, `ParamSpec`, `Concatenate`, `TypeVarTuple`, `TypedDict`, `NamedTuple`, `Annotated`, `override`, `final`, `Pattern` and `Match` (from `re`), and `TracebackType` (from `types`). Test in `tests/test_rules_imports.py` style: each name used in an annotation gets its import inserted, an existing import is not duplicated, a local shadow is aliased. Report any emitter that still hand-rolls `insert_import` instead of `ensure_names`.

Steps: *to be written*

Done when: *to be written*

H-09 — Gate for the conversion pair (P-09)

- **Status:** open
- **Priority:** P1
- **Area:** Testing / QA
- **Readiness:** needs recipe

Context: Store the accepted numbers of `python -m benchmarks.pair` as `benchmarks/pair_baseline.json`; add

Context: Store the accepted numbers of `python -m benchmarks.pair` as `benchmarks/pair_baseline.json`; add `python -m benchmarks.pair ... --gate BASELINE` that exits 1 on lower exact recall or any new contradiction; mirror how `benchmarks/report.py` gates the recovery baseline. Extend `tests/test_bench_pair.py`.

Steps: *to be written*

Done when: *to be written*

H-11 — `astart fix --jobs N` (X-06, M7-04/05): one process per file in `cli/commands.py` with…

- **Status:** open
- **Priority:** P1
- **Area:** Backend
- **Readiness:** needs recipe

Context: `astart fix --jobs N` (X-06, M7-04/05): one process per file in `cli/commands.py` with `ProcessPoolExecutor`, results printed and written in input order, a crash in one file reported for that file only (exit 2), `--jobs 1` identical to today. Not with `--project` (needs one index): refuse the combination with a clear message. Tests `tests/test_cli_jobs.py`: same bytes as `--jobs 1` on three files, order stable, one unparsable file does not stop the others.

Steps: *to be written*

Done when: *to be written*

H-10 — Differential oracle (T-07)

- **Status:** open
- **Priority:** P2
- **Area:** Testing / QA
- **Readiness:** needs recipe

Context: New `tests/test_negotiation_oracle.py` using `pytest.importorskip("typing_utils")` (add it to the dev extras in `pyproject.toml` only if a `dev` extra already exists): for 40 pairs of builtin/`typing` generics evaluate `typing_utils.issubtype` and compare with `recovery.relation(written=a, truth=b)` (subtype ⇒ `None` / `narrower`) and with `TypeAlgebra().intersect(left=b, right=a)` (a subtype of b ⇒ result `a`). Disagreements are listed under the task, not "fixed" in the tests.

Steps: *to be written*

Done when: *to be written*

H-12 — Keyword style controls (M6-04): `min_positional` (leave the first N arguments positional)…

- **Status:** open
- **Priority:** P2
- **Area:** Backend
- **Readiness:** needs recipe

Context: Keyword style controls (M6-04): `min_positional` (leave the first N arguments positional) and `[tool.astart.params]` `keyword-skip = ["len", "print"]` (callees never keyworded). Plumb through `FixSettings` → `rules/facts.py` `keyword_candidates` (read how `kept_positionals` is applied). Tests in `tests/test_rules_keyword_style.py`; default behaviour unchanged (run the danger matrix `tests/test_rules_kwargs_danger.py`).

Steps: *to be written*

Done when: *to be written*

14. Implementation recipes

Scope: Step-by-step recipes for tasks above.

The rule for this section: **one step = one edit or one command, with the file and the check**. Read the step, do it, run its check, go to the next. The task IDs are the ones in the milestone sections above; when a task here is done, set the original task to `done` too. Batches still to be decomposed are listed in 20.9.

M7-04 + M7-05 — parallel `astart fix --jobs N` (one feature, two IDs)

Facts: `run_fix` (src/astart/cli/commands.py:809) loops `for path in found.files: outcome = _fix_file(...)`. `FixOutcome` is a frozen dataclass of `Path` / `str` / `tuple`, so it pickles. `Catalog`, `Checker` and `FixCache` do not pickle, so every worker builds its own. Project mode (`--project`) shares one index: keep it serial.

1. **Setting.** In `FixCommandSettings` (commands.py:138) add this line to the `Attributes:` docstring: `jobs: Worker processes; 1 means serial.` and the field `jobs: int = 1` after `no_docs`. Check: `python -c "from astart.cli.commands import FixCommandSettings"`.
2. **Flag.** In `src/astart/__main__.py`, beside the other `fix` flags (search for `--min-confidence`), add `command.add_argument("--jobs", type=int, default=1)`; where `FixCommandSettings(...)` is built pass `jobs=max(1, args.jobs)`. Add

`command.add_argument("--jobs", type=int, default=1)`, where `FixCommandSettings(...)` is built to pass `jobs=max(1, args.jobs)`. Add the flag to the README flag table (tests/test*readme*flags.py fails otherwise).

3. **Module-level worker.** Above `run_fix` add (it must be module level to pickle): ```python _WORKER: dict[str, object] = {}

   def *init*worker(settings: FixCommandSettings) -> None: """Build the per-process catalog and checker once.""" *WORKER["catalog"] = load*default*catalog(registries=default*registries()) _WORKER["checker"] = *make*checker(settings=settings)

   def *fix*in_worker(path: Path, settings: FixCommandSettings) -> FixOutcome: """Fix one file with the worker's catalog (no project context).""" return *fix*file(path=path, settings=settings, catalog=*WORKER["catalog"], checker=*WORKER["checker"], project=*ProjectContext(index=None, libraries=None, cache=None))*
   ``` *The project rules forbid object annotations: type* WORKER `as a small` @final `class` _Worker `with two attributes instead (` catalog: Catalog | None `,` checker: Checker | None`) and a module instance. Do that, not the dict.

4. **Dispatch.** In `run_fix` replace the loop header with a helper `outcomes = _outcomes(found.files, settings, ...)` returning an iterator: serial `(_fix_file(...) for path in files)` when `settings.jobs == 1` or `index is not None`; otherwise `python with ProcessPoolExecutor(max_workers=settings.jobs, initializer=_init_worker, initargs=(settings,)) as pool: yield from po` (`from concurrent.futures import ProcessPoolExecutor`, `from itertools import repeat`). `pool.map` yields in input order, so the report text is identical to serial. This is the "streaming" of M7-05: each outcome is written as soon as it arrives.

5. **Cache and `--check`.** The on-disk cache (`_make_cache`) is not used in workers (cache=None). Say so in the flag help text. `--check` / `--diff` modes work unchanged because the parent writes/prints.

6. **Tests** (`tests/test_cli_jobs.py`): (a) write 6 small untyped files to `tmp_path`; run `main(["fix","--diff","--jobs","1",...])` and `--jobs 2`; assert identical stdout and exit code; (b) a file with a syntax error under `--jobs 2` yields `error:` for that path and exit 2 while the others are still reported; (c) `--jobs 0` is treated as 1. Mark (a) `slow` if it takes more than 5 s. Done when: the three tests pass, FAST is green, and `--jobs 4` on `tests/corpus_ext` prints the same summary line as `--jobs 1`.

M7-06 — memory budget tests

1. Create `tests/test_memory_budget.py`, marker `slow`.
2. Helper `_big_module(functions: int) -> str` that repeats a 12-line untyped function (`def f{i}(a, b=1):\n x = a + b\n return x\n`) `functions` times.
3. Test body: `tracemalloc.start()`; call `fix_source(source=..., filename="big.py", catalog=..., registries=..., settings=FixSettings(...))`; `_, peak = tracemalloc.get_traced_memory()`; `tracemalloc.stop()`.
4. Assert `peak < 400 * 1024 * 1024` for 2000 functions (≈ 24k lines). Record the measured value in the test docstring; set the bound to 3× the measured value.
5. Add a second test that the peak for 4000 functions is less than 2.5× the peak for 2000 (linear, not quadratic). Done when: both tests pass; the numbers are written in TODO under M7-06.

M7-11 — pre-commit and CI recipe

Facts: `.pre-commit-hooks.yaml` has one hook `astart-fix` (`astart fix --check`). `.github/workflows/ci.yml` is the project's own CI. 1. Add a second hook to `.pre-commit-hooks.yaml`:
`yaml - id: astart-check name: astart check entry: astart check language: python types: [python]` 2. Create `docs/ci.md` with three fenced examples: a `.pre-commit-config.yaml`
(`repos: - repo: https://github.com/<owner>/astart rev: v0.1.0 hooks: [{id: astart-fix, args: [--min-confidence, "0.8"]}]`), a GitHub Actions step (`pip install astart && astart fix --check --min-confidence 0.8 src`), and the exit-code table (0 clean, 1 would change, 2 usage error). 3. Link `docs/ci.md` from README under the flags table. 4. `tests/test_docs_ci.py`: read `.pre-commit-hooks.yaml` as text; assert it contains `id: astart-fix` and `id: astart-check`; assert `docs/ci.md` mentions every `id:` found. (Do not import yaml.) Done when: the test passes.

M7-13 — `astart explain CODE path:line[:col]`

Facts: `run_explain(code, out, err)` (commands.py:537) prints a rule's rationale and examples. The new form explains one finding. 1. Read `src/astart/rules/reasons.py` and `run_check` (commands.py:432) first; note how a finding stores its rule code, line and column. 2. In `__main__.py` give the `explain` parser an optional second positional: `explain.add_argument("location", nargs="?")`. 3. In commands.py add `run_explain_at(*, code, location, out, err) -> int`: split `location` with `rpartition(":")` twice into `path`, `line`, optional `column`; on `ValueError` write `astart: bad location` and return `Constants.usage_error`. 4. Inside it call the same check path as `run_check` for that one file (`_check_file`, commands.py:393), then keep findings whose `rule.code == code` and whose `line` matches (and `column` when given). 5. For each match print: the rule header (reuse the lines of `run_explain`), the finding's message, and its reasons (the evidence list from `reasons.py`). If none match print `astart: no <code> finding at <location>` and return 1. 6. Dispatch:
`if args.command == "explain": return run_explain(...) if args.location is None else run_explain_at(...)`. 7. Tests
(`tests/test_cli_explain_at.py`): file with one TYP001 finding; matching location prints the message; wrong line returns 1; `bad` location returns 2.

M7-14 — `--backup` and `astart undo`

Facts: `_write_atomic(path, text, encoding)` (commands.py:564) already writes safely. Design (decided): backups live in `.astart-backup/<run-id>/` next to the working directory; one `journal.json` per run; undo never overwrites a file edited since. 1. New module `src/astart/cli/backup.py` with `@final @dataclass(frozen=True, slots=True) class Entry: path: str; before_sha: str; after_sha: str; saved: str`. 2.
`def save(*, run_dir: Path, path: Path, before: str, after: str, encoding: str) -> Entry`: write `before` to
`run_dir / f"{index:05d}.bak"` with `_write_atomic`; compute `sha256(text.encode(encoding))` for both texts; return the Entry. 3.
`def write_journal(*, run_dir: Path, entries: Sequence[Entry]) -> None` using
   ```

`def write_journal(*, run_dir: Path, entries: Sequence[Entry]) -> None` using `json.dumps({"entries": [asdict(e) for e in entries]}, indent=2)`. 4. `def undo(*, run_dir: Path, out: TextIO) -> int`: for each entry, read the current file; if its sha equals `after_sha` restore `saved` content with `_write_atomic`; otherwise print `skipped (edited since): <path>`. Return 1 if any skipped, else 0. 5. Flag: `fix --backup` (default off). In `run_fix` where the file is written (`_emit`), call `backup.save` first when `settings.backup`. Run id = `datetime.now().strftime("%Y%m%dT%H%M%S")`. Add `backup: bool = False` to `FixCommandSettings` (+ docstring). 6. Subcommand `astart undo [RUN_ID]`: no id → newest directory in `.astart-backup`. 7. Tests (`tests/test_cli_backup.py`): fix a file with `--backup`, undo, text is the original; edit after fix then undo → file untouched and exit 1.

M7-15 — wheel and entry-point check

Facts: `pyproject.toml` already declares `[project.scripts] astart` and `package-data` for the catalog JSON and `py.typed`; `dist/` exists. 1. Create `scripts/check_wheel.py` that (a) runs `python -m build --wheel --outdir <tmp>`, (b) opens the wheel with `zipfile`, (c) asserts it contains `astart/py.typed`, `astart/rules/schema.json`, at least one `astart/rules/catalog/*.json`, and `astart/stubs/__init__.py`, (d) creates a venv in `<tmp>` and runs `<venv>/bin/astart rules`, asserting exit code 0 and that the output has more than 10 lines. 2. Run it once; fix `package-data` / `MANIFEST.in` for whatever is missing (new packages `negotiation` and `stubs` are found by `packages.find include`). 3. Add a CI step in `.github/workflows/ci.yml`: `python scripts/check_wheel.py`. Done when: the script prints `ok` locally.

M8-14 — maturity decides defaults

Facts: `Maturity` (vocabulary.py:104) has `experimental/stable/deprecated`; every catalog rule has the field (model.py:138). 1. Find where rules are selected for a run: `grep -n "catalog.rules" src/astart/cli/commands.py src/astart/rules/engine.py`. 2. Add `FixSettings.experimental: bool = False` and `CheckSettings.experimental` with docstring lines; CLI flag `--experimental` (+ README row). 3. In the selection helper skip rules whose `maturity is Maturity.experimental` unless the setting is on. 4. For `deprecated` rules still run but print one line per run `note: rule <code> is deprecated`. 5. Tests: add a temporary rule with `maturity: experimental` in a test catalog (see `tests/test_rules_catalog.py` for building one); default run does not report it, `--experimental` does. Done when: all 3 tests pass and `astart rules` still lists every rule.

M5 protocol follow-ups (all in `src/astart/rules/protocols.py`)

Facts: `Member(name, is_method, positional, keywords, returns, is_async)`; `_collect` (line 374) walks uses of a parameter; `_member_source` (517) renders a member; `protocol_source` (576) renders the class; generated protocols use `Protocol` only (no decorator).

M5-06 read-only vs writable attributes 1. Add `writable: bool = False` to `Member` (docstring line too). 2. In `_collect`, where an `Attribute` on the parameter is found, check `isinstance(node.ctx, Store)` (also `AugAssign` target and `del`); record `writable=True` for that member name (merge with `any`). 3. In `_member_source`, for `not member.is_method and member.writable`: a property would be wrong (it forbids assignment). Render `{pad}{name}: {annotation}` when `member.returns` is known; when unknown, return `None` from the usage builder so that **no protocol is made** (soundness over coverage). 4. Tests in `tests/test_rules_protocols.py`: read-only attribute → `@property`; assigned attribute with a provable type → plain annotation; assigned attribute with unknown type → no fix.

M5-07 `runtime_checkable` (decision: never emitted by default) 1. Add `FixSettings.runtime_protocols: bool = False`. 2. Only when on AND the module contains `isinstance(<expr>, <ProtocolName>)` for a protocol this run creates, prepend `@runtime_checkable` and make `ensure_names` import it (`known_imports` already lists `runtime_checkable` → `typing`). 3. Test both settings; default output has no decorator.

M5-11 per-function placement switch 1. Config key `[tool.astart] protocol-placement = "module" | "function"` read in `cli/config.py` (copy the pattern of an existing string key). 2. `module` is today's behaviour. `function` puts the class definition directly above the def that uses it (still module level, never nested). 3. Test: two functions, switch flips the order of class and def.

M5-12 `TYPE_CHECKING` placement 1. Only for protocols used in annotations only (parameters never instantiated). 2. Wrap the class in `if TYPE_CHECKING:` and quote the annotation (`'Proto'`); `ensure_names` adds `from typing import TYPE_CHECKING`. 3. Test: output imports cleanly (`exec` the result) and the annotation is a string.

M5-20 `TypeVarTuple` pass-through; M5-21 variance — both are suggest-only. 1. Add the detection to `typevars.py` as a pure function returning `None` unless exactly the pattern holds: `*args` returned unchanged as a tuple (M5-20) / a TypeVar only read (covariant) or only written (contravariant) inside a generic class (M5-21). 2. Gate behind `--experimental` (see 20.7); no write until a gold case exists. 3. One positive and two refusal tests each.

Still to be decomposed (next batches, by priority)

The remaining open tasks that are one-liners today, in the order to expand: 1. M4 precision: M4-01, M4-03, M4-06 (see W-13), M4-08, M4-13, M4-16..19. 2. M2 OOP: M2-05, M2-10, M2-12, M2-13 (exit gate). 3. M6 kwargs: M6-02, M6-04, M6-09. 4. M5 remainder: M5-04, M5-09, M5-10, M5-19, M5-23, M5-25. 5. M7 rest: M7-10 (LSP), M7-12 (interactive review). 6. M8 / CC: M8-11, M8-13, CC-01, CC-02, CC-03. 7. F-16 and the 25 open F-tasks in 18.12 (already have steps; check each has a test). Expand each with the same format as 20.1–20.8: facts (file:line), numbered atomic steps with a snippet where the code is non-obvious, tests, done-when.